

Machine Learning

Semester 1, 2024

Dr Simon Denman

Tarang Janawalkar

This work is licensed under a Creative Commons
“Attribution-NonCommercial-ShareAlike 4.0 International” license.



Contents

Contents	1
1 Introduction	4
1.1 Machine Learning Process	4
1.2 Machine Learning Paradigms	4
1.3 Importance of Data	5
1.4 Data Splitting	5
1.5 Traditional Machine Learning to Deep Learning	6
1.5.1 Traditional Machine Learning	6
1.5.2 An Aside on Neural Networks	6
1.5.3 Deep Learning	6
I Supervised Learning	7
2 Linear Regression	7
2.1 The Loss Function	7
2.2 Correlation	8
2.3 Simple Linear Regression	9
2.4 Multiple Linear Regression	10
2.5 Probabilistic Interpretation	11
2.6 Assumptions of Linear Regression	12
2.7 Testing the Assumptions of Linear Regression	12
2.7.1 Linearity	12
2.7.2 No Multicollinearity	12
2.7.3 Independence	13
2.7.4 Normality	13
2.7.5 Homoscedasticity	13
2.7.6 Exogeneity	13
2.8 Testing Overall Model Fit	13
2.8.1 p-Values	16
3 Regularisation	16
3.1 Bias and Variance	16
3.2 Regularisation Techniques	17
3.3 LASSO Regularisation	18
3.4 Ridge Regularisation	18
3.5 Ridge vs LASSO Regularisation	19
4 Classification	20
4.1 Logistic Regression	20
4.1.1 Multiclass Logistic Regression	21
4.2 Support Vector Machines	23
4.2.1 Hard Margin SVM	23

4.2.2	The Dual Problem	25
4.2.3	Kernel Methods	26
4.2.4	Common Kernels	27
4.2.5	Soft Margin SVM	28
4.3	K-Nearest Neighbours	30
4.4	Random Forest	30
4.4.1	Decision Trees	31
4.4.2	Random Forests	31
4.5	Multiclass Classification	31
4.6	Evaluating a Classification Model	32
4.7	Model Selection	33
II	Deep Learning	33
5	Feedforward Neural Networks	33
5.1	Fully Connected Networks	33
5.1.1	Perceptron	33
5.1.2	Multilayer Perceptron	34
5.1.3	Vectorised Form	35
6	Training Neural Networks	36
6.1	Loss Functions	36
6.2	Activation Functions	38
6.3	Backpropagation	39
6.4	Vectorised Backpropagation	41
6.5	Batch Backpropagation	41
6.6	Considerations for Classification Problems	43
6.6.1	Target Variable Encoding	43
6.6.2	Output Layer	43
6.7	Ensemble Methods	43
7	Optimisation Algorithms and Regularisation	43
7.1	Gradient Descent	44
7.2	Stochastic Gradient Descent	44
7.2.1	Epochs	44
7.2.2	Batch Size	45
7.3	Optimisers	45
7.3.1	Momentum	45
7.3.2	Nesterov Accelerated Gradient (NAG)	45
7.3.3	Resilient Propagation (RProp)	46
7.3.4	Adaptive Gradient (AdaGrad)	46
7.3.5	RMSprop	46
7.3.6	Adam	46
7.4	Regularisation	46
7.4.1	Dropout Layers	47
7.4.2	Batch Normalisation	47

7.4.3	Weight Regularisation	47
7.4.4	Fine Tuning	47
7.4.5	Augmentation	47
8	Convolutional Neural Networks	48
8.1	Convolutional Layers	48
8.2	Pooling Layers	48
8.3	Deep Convolutional Neural Networks	49
8.3.1	VGG Networks	49
8.3.2	Residual Networks	49
III	Unsupervised Learning	49
9	Clustering	49
9.1	K-Means Clustering	50
9.2	Gaussian Mixture Models	50
9.3	Model Selection	50
10	Dimensionality Reduction	51
10.1	Principal Component Analysis	51
10.2	Linear Discriminant Analysis	51
10.3	t-Distributed Stochastic Neighbour Embedding	51
11	Autoencoders	51
12	Sequences and Attention Mechanisms	51
A	Numerical Summaries of Data	52
A.1	Mean	52
A.2	Variance	52
A.3	Covariance	52

1 Introduction

Machine learning is a field of computer science concerned with the development of statistical algorithms that enable computer systems to learn insights from data. Machine learning is a multi-disciplinary field that is related to statistics, pattern recognition, data mining, and artificial intelligence.

1.1 Machine Learning Process

Broadly speaking, there are three steps to enable machines to learn.

1. **Input Data:** A model must be provided with input data. This data consists of audio, images, text, or any other form of data, which enables us to build a model. Ideally, it is desirable to have a large amount of data, as this produces a more accurate model, and removes the possibility of overfitting when many features are present.
2. **Data Abstraction:** To effectively train a model on this data, it needs to be abstracted into a consistent format that can be understood by a model. This includes pre-processing the data, removing any noise or irrelevant features, and ensuring that the dimensionality (or the number of features) is the same across all samples.
3. **Generalisation:** When data is ready, we can use a machine learning technique to determine whether a model generalises well to new data.

1.2 Machine Learning Paradigms

There are three main types of machine learning paradigms:

- **Supervised Learning:** In this type of learning, a model is provided both input and expected output data. The purpose here is to learn a mapping from input to output, to predict an output for new unseen data.

For example, we might provide a model a video of pedestrians with a person count in each frame. The model then regresses features such as size, texture, and shape to predict the number of people in unseen frames.

Another related example may include object tracking, where a model learns to track objects in a video, given a pre-labelled set of frames with tracked objects.

- **Unsupervised Learning:** In this type of learning, a model is provided only input data, and is mainly used for knowledge discovery—to find order and characteristics in data.

An example of this is clustering, where a model might group people who are walking together, based on their motion characteristics.

Another example is the detection of anomalies or abnormal behaviour in data. For example, a model may be trained on pedestrian motion, and use unsupervised learning to track vehicles and bicycles in a pedestrian zone.

- **Reinforcement Learning:** In this type of learning, a model learns to make decisions by interacting and responding to an environment. The model is rewarded or penalised based on its actions, and its goal is to learn the best sequence of actions to maximise its reward.

An example of this may be a robot learning to walk. The objective might be to maximise the distance travelled in one direction.

1.3 Importance of Data

Data is the most important aspect of machine learning, both in terms of **quality and quantity**. A high-quality dataset that is well prepared and which covers all possible cases considered in a model, leads to better outcomes.

- a model cannot produce predictions that are better than the data it is trained on
- a model cannot compensate for errors in the annotation of data

It is also important to consider **data diversity** to ensure that a model does not learn **biases** present in the data. For example, a male dominated dataset may lead to a model that is biased against women.

Data also suffers from the *curse of dimensionality*. Datasets often have:

- a large number of dimensions or features (i.e., columns, observed variables)
- but a small number of samples (i.e., rows, observations)

In such cases, the feature space is sparsely populated and the model may **overfit**. Every new feature increases the complexity of the model, and also necessitates more data, to produce an accurate model—this number depends on the type of classifier or learning algorithm used, and therefore a model should be kept simple.

1.4 Data Splitting

To efficiently train and evaluate a model, we can split the data into three sets:

- **Training Set:** This set is used to train the model.
- **Validation Set:** This set is used to tune model hyperparameters to evaluate a model's performance. The training set will be trained using different hyperparameters to evaluate which set of hyperparameters produce the best model.
- **Test Set:** This set is used to evaluate the model's performance on unseen data.

This approach uses *holdout validation* where data is *held out* for validation and/or testing, so that the training, validation, and testing stages are completely separate. Note that this requires datasets to be large enough to ensure that a model has enough data for training.

When **insufficient data** is available, *cross-validation* can be used to dynamically split the dataset into training and validation sets (i.e., a 80% training/20% validation split). This may be repeated 5 times, so that each split is used as a validation set. The best model is then selected based on the average performance across all splits.

In some cases, machine learning may not be possible if the dataset is too small. This may be because the data does not contain the patterns and relationships that we are trying to learn. For some problems, this threshold may be a few hundred samples, while for others, it may be a few million. Extrapolation beyond the data may also be problematic in such cases.

1.5 Traditional Machine Learning to Deep Learning

1.5.1 Traditional Machine Learning

The traditional machine learning pipeline typically consists of:

- **Pre-processing:** Preparing data for a model using normalisation, scaling, noise reduction, etc.
- **Feature Extraction:** Extracting features from data; MFCC (audio), HOG (image/video), and bag-of-words (text).

This step is often informed by the task. For example, if we need to recognise shapes in images, we might use a technique that captures edge detection. In the case of audio, we might use a technique that captures frequency information.

- **Machine Learning:** Passing the features to a model using a machine learning technique.

In this model, features are **hand-crafted**, and based on domain-specific knowledge. Choosing the optimal features for a given problem can be a tedious and iterative process. This leads to different formulations for different tasks, and may significantly increase the complexity of a problem, especially when we need to extract multiple sets of features.

1.5.2 An Aside on Neural Networks

A neural network is modelled after the human brain and nervous system. It is built on a single unit or **node** called a *neuron*, in which data is passed through a series of **layers**, each of which is connected to other neurons through **edges**. Neural networks typically consist of a large number of parameters and many interconnections.

Choosing the appropriate number of layers in a neural network (or the *depth* of a neural network), is problem dependent. A higher depth allows us to learn *higher level features*, and contains more parameters, but this is often harder to learn and also increases the likelihood of overfitting. Larger networks also require more data, and therefore more memory and computational resources.

1.5.3 Deep Learning

Deep learning is a subset of machine learning that uses neural networks to learn features from data. It differs from the traditional machine learning pipeline in that it does not require manual feature extraction, but instead learns its own representation from pre-processed data. This makes deep learning more **adaptable** to different tasks, and also reduces the complexity of a problem.

A downside of deep learning is that models are often extremely large compared to traditional approaches;

- a traditional machine learning model may have a few hundred parameters
- a deep learning model may have millions of parameters

This leads to an increase in computational resources required to train and evaluate a model, and also makes a model difficult to interpret.

In deep learning, feature engineering is replaced with **network engineering**, where we must carefully choose the appropriate architecture for a problem. This includes:

- the number of layers,
- the number of neurons in each layer, and
- the type of activation function used.

This is often an iterative process, and it may not be feasible to find the optimal architecture for a given problem, through exhaustive testing.

Part I

Supervised Learning

2 Linear Regression

A simple model for predicting the relationship between a dependent variable y (**response**), and an independent variable x (**predictor**), is a straight line. Given a set of n input-output pairs $(x^{(i)}, y^{(i)})$, we can model a linear relationship between x and y as:

$$y^{(i)} = \beta_0 + \beta_1 x^{(i)}$$

where β_0 is the **y -intercept** and β_1 is the **gradient** of the line. In practice, data rarely fits a model perfectly and therefore, we will assume the relationship:

$$\hat{y}(x) = \hat{\beta}_0 + \hat{\beta}_1 x$$

where $\hat{y}$ is known as the **hypothesis function** which **predicts** value of y , and $\hat{\beta}_0$ and $\hat{\beta}_1$ are the **estimated** values of the **parameters** β_0 and β_1 , respectively.

2.1 The Loss Function

Consider a **residual** term ϵ to account for the error in each observation:

$$\begin{aligned} y^{(i)} &= \hat{y}(x^{(i)}) + \epsilon^{(i)} \\ &= \hat{\beta}_0 + \hat{\beta}_1 x^{(i)} + \epsilon^{(i)} \end{aligned}$$

where ϵ is assumed to be normally distributed with zero mean and σ^2 variance¹:

$$\epsilon \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2).$$

The values of $\hat{\beta}_0$ and $\hat{\beta}_1$ arise when we consider the **loss function**²:

$$L(\beta) = \frac{1}{2} \epsilon^2 = \frac{1}{2} (y - \hat{y})^2 = \frac{1}{2} (y - (\beta_0 + \beta_1 x))^2$$

¹Note that as y depends on ϵ , we can also show that $y \sim \mathcal{N}(\beta_0 + \beta_1 x, \sigma^2)$ where y is normally distributed for fixed values of x .

²To see how this loss function is derived, see Section 2.5

which we wish to minimise. As these estimators apply to the entire training set, we will consider the **cost function**:

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n L(\beta).$$

which averages the loss across all observations. The goal of this cost function is to find the best parameters $\hat{\beta}$ that minimise the loss across all observations:

$$\hat{\beta} = \arg \min_{\beta} J(\beta).$$

In this model, we must ensure that there are at least as many observations as there are parameters to estimate.

2.2 Correlation

Correlation is a statistical relationship between two variables. This measure allows us to determine the strength of a linear relationship between two variables. One common measure of correlation is the **Pearson correlation coefficient**, which is given by:

$$\rho_{xy} = \frac{\sigma_{xy}}{\sigma_x \sigma_y}$$

where σ_{xy} is the **covariance** between x and y , and σ_x and σ_y are the variances of x and y , respectively. Using the sample covariance³:

$$r_{xy} = \frac{s_{xy}}{s_x s_y}.$$

The sample covariance s_{xy} has the following characteristics:

- $s_{xy} > 0$: y increases as x increases
- $s_{xy} < 0$: y decreases as x increases
- $s_{xy} \approx 0$: no linear relationship between x and y

To generalise this measure across datasets, we often consider the *normalised* correlation coefficient r_{xy} , which tells us the strength of a linear relationship between two variables. The following diagram shows various datasets and their correlation coefficients:

³See Appendix A for more details.

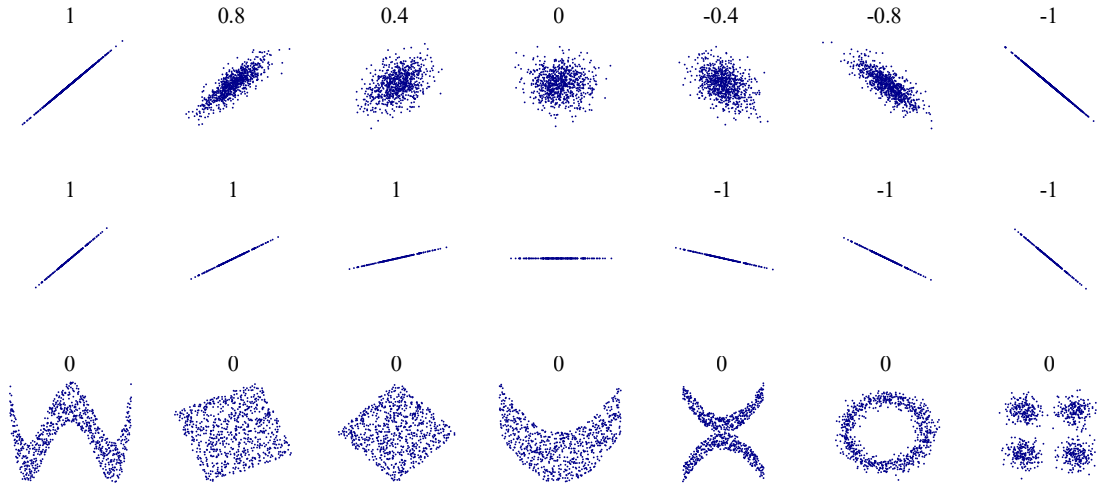


Figure 1: Pearson correlation coefficient of x and y for several sets of points.

There are a number of important points to note about the correlation coefficient:

- The correlation coefficient does not contain any information about the gradient, as is seen in the second row.
- A correlation coefficient of 0 indicates *no linear relationship*, it does not imply that there is *no relationship*, as is seen in the third row.
- The correlation coefficient does not imply *causation*. Care must be taken when assuming that a relationship between two variables is causal.

To minimise redundancy in a model, it is crucial for predictors to be correlated with response variables, and uncorrelated to other predictors.

2.3 Simple Linear Regression

Simple linear regression is a special case of linear regression, where we have only one input variable x_1 , and we can use the method of maximum likelihood estimation to estimate the parameters β_0 and β_1 :

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x^{(i)} - \bar{x})(y^{(i)} - \bar{y})}{\sum_{i=1}^n (x^{(i)} - \bar{x})^2}$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

where $\bar{x}$ and $\bar{y}$ are the sample means of x and y , respectively.

2.4 Multiple Linear Regression

In multiple linear regression, we consider a p -dimensional feature space, or p input variables, with $p + 1$ parameters to estimate. Here hypothesis function generalises to:

$$\hat{y}(x) = \hat{\beta}_0 x_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p = \sum_{i=0}^p \hat{\beta}_i x_i = \boldsymbol{\beta}^\top \mathbf{x}.$$

where we have introduced the feature $x_0 := 1$ to simplify our notation. There are several ways to estimate $\hat{\boldsymbol{\beta}}$, and here we will use matrix calculus. The cost function $J(\boldsymbol{\beta})$ can be written as:

$$J(\boldsymbol{\beta}) = \frac{1}{2n} \sum_{i=1}^n (y^{(i)} - \boldsymbol{\beta}^\top \mathbf{x}^{(i)})^2 = \frac{1}{2n} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 = \frac{1}{2n} \boldsymbol{\epsilon}^\top \boldsymbol{\epsilon}.$$

where $\mathbf{X}$ is an $n \times (p + 1)$ matrix of input variables:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \cdots & x_p^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \cdots & x_p^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} & \cdots & x_p^{(n)} \end{bmatrix}$$

Consider the vector derivative of the cost function with respect to $\boldsymbol{\beta}$:

$$\begin{aligned} \frac{\partial J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} &= \frac{1}{2n} \frac{\partial}{\partial \boldsymbol{\beta}} [\boldsymbol{\epsilon}^\top \boldsymbol{\epsilon}] \\ &= \frac{1}{2n} \frac{\partial}{\partial \boldsymbol{\beta}} [(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})] \\ &= \frac{1}{2n} \frac{\partial}{\partial \boldsymbol{\beta}} [(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \mathbf{y} - (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \mathbf{X}\boldsymbol{\beta}] \\ &= \frac{1}{2n} \frac{\partial}{\partial \boldsymbol{\beta}} [\mathbf{y}^\top \mathbf{y} - (\mathbf{X}\boldsymbol{\beta})^\top \mathbf{y} - \mathbf{y}^\top \mathbf{X}\boldsymbol{\beta} + (\mathbf{X}\boldsymbol{\beta})^\top \mathbf{X}\boldsymbol{\beta}] \\ &= \frac{1}{2n} \frac{\partial}{\partial \boldsymbol{\beta}} [\mathbf{y}^\top \mathbf{y} - \boldsymbol{\beta}^\top (\mathbf{X}^\top \mathbf{y}) - (\mathbf{y}^\top \mathbf{X}) \boldsymbol{\beta} + \boldsymbol{\beta}^\top (\mathbf{X}^\top \mathbf{X}) \boldsymbol{\beta}] \\ &= \frac{1}{2n} \left(-\mathbf{X}^\top \mathbf{y} - (\mathbf{y}^\top \mathbf{X})^\top + 2\mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} \right) \\ &= \frac{1}{n} (\mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} - \mathbf{X}^\top \mathbf{y}). \end{aligned}$$

This implies that the optimal parameters $\boldsymbol{\beta}$ are given by:

$$\begin{aligned} \frac{\partial J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} &= \mathbf{0} \\ \mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} - \mathbf{X}^\top \mathbf{y} &= \mathbf{0} \\ \mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} &= \mathbf{X}^\top \mathbf{y} \\ \boldsymbol{\beta} &= (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \end{aligned}$$

These are known as the **normal equations**, and they provide the optimal parameters $\hat{\boldsymbol{\beta}}$ that minimise the cost function $J(\boldsymbol{\beta})$.

2.5 Probabilistic Interpretation

To understand why we use the loss function $L(\beta)$, consider the probability density function of the residuals ϵ :

$$p(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\epsilon^{(i)2}}{2\sigma^2}\right),$$

which implies

$$p(y^{(i)} | \mathbf{x}^{(i)}; \beta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \beta^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right).$$

where the probability is conditioned on the input $\mathbf{x}$ and parametrised by β . If we treat this as a function of β , we can use the **likelihood function** $\mathcal{L}$ to consider the probability of observing a response y given an input $\mathbf{x}$

$$\mathcal{L}(\beta) = p(\mathbf{y} | \mathbf{X}; \beta)$$

which we wish to maximise. As we assume all residuals to be independently distributed, we can take the product of all observations:

$$\mathcal{L}(\beta) = \prod_{i=1}^n p(y^{(i)} | \mathbf{x}^{(i)}; \beta) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \beta^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right).$$

Now, rather than maximising the likelihood function, we can also maximise any monotonically increasing function of $\mathcal{L}$, to simplify our calculations. Let us consider the **log-likelihood** function ℓ :

$$\begin{aligned} \ell(\beta) &= \ln \mathcal{L}(\beta) \\ &= \ln \left[\prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \beta^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right) \right] \\ &= \sum_{i=1}^n \ln \left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \beta^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right) \right] \\ &= \sum_{i=1}^n \ln \left[\frac{1}{\sqrt{2\pi\sigma^2}} \right] - \sum_{i=1}^n \frac{(y^{(i)} - \beta^\top \mathbf{x}^{(i)})^2}{2\sigma^2} \\ &= -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{\sigma^2} \sum_{i=1}^n \frac{1}{2} (y^{(i)} - \beta^\top \mathbf{x}^{(i)})^2 \\ &= -\frac{n}{\sigma^2} \left[\frac{\sigma^2}{2} \ln(2\pi\sigma^2) + \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y^{(i)} - \beta^\top \mathbf{x}^{(i)})^2 \right]. \end{aligned}$$

Hence, maximising $\ell(\beta)$ is equivalent to minimising the cost function $J(\beta)$,

$$J(\beta) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y^{(i)} - \beta^\top \mathbf{x}^{(i)})^2$$

from which we can define the loss function $L(\beta)$:

$$L(\beta) = \frac{1}{2}(y - \beta^\top \mathbf{x})^2.$$

This technique is known as **maximum likelihood estimation**.

2.6 Assumptions of Linear Regression

There are several assumptions that must be satisfied for linear regression to be valid:

1. **Linearity**: The relationship between the response and predictors is linear.
2. **No Multicollinearity**: Predictors are linearly independent.
3. **Independence**: Responses are linearly independent.
4. **Normality**: Residuals are normally distributed.
5. **Homoscedasticity**: Residuals have constant variance and do not depend on i .
6. **Exogeneity**: There is no correlation between predictors and residuals.

2.7 Testing the Assumptions of Linear Regression

To test the above assumptions, we can use the following techniques:

- Plots of predictions and actual values: Compare the regression with actual values
- Regression outputs: R^2 , adjusted R^2 , F-statistic, RMSE (root mean squared error)
- Correlation: Correlation between pairs of predictors, and between the response and predictors
- Analysis of residuals: Testing the normality and variance of residuals

2.7.1 Linearity

- **Scatter plot**: Plot responses with the predicted model to visually check for linearity
- **Residual plot**: Plot residuals against predictors. If the residuals are randomly distributed about 0, then the model suggests linearity

2.7.2 No Multicollinearity

- **Correlation matrix**: Check for high correlation between predictors, i.e., $|r_{x_1, x_2}| > 0.7$

High correlation between predictors may result in unreliable p -values. In this case, it might be necessary to remove one of the predictors from the model.

2.7.3 Independence

It is important to verify that responses are independent of each other. In time series data, this assumption can be tested using the **Durbin-Watson** test, which tests for the presence of **autocorrelation** in the residuals. A value close to 2 suggests no autocorrelation, while a value close to 0 or 4 suggests positive or negative autocorrelation, respectively.

2.7.4 Normality

- **Q-Q plot:** A quantile-quantile plot can be used to compare the distribution of residuals to a normal distribution. In this plot, we should expect the residuals to lie on a straight line.
- **Histogram:** A histogram of residuals can also be used to check for normality.

2.7.5 Homoscedasticity

- **Residual plot:** Plot residuals against predictions. A constant spread of residuals about 0 suggests homoscedasticity.

2.7.6 Exogeneity

- **Residual plot:** Plot residuals against predictors. If the residuals are randomly distributed about 0, then the model suggests exogeneity.

2.8 Testing Overall Model Fit

The OLS (Ordinary Least Squares) model in the `statsmodels` package can be used to produce regression outputs from a linear regression model. The following table is an example of the output from a linear regression model:

Dep. Variable:	y	R-squared:	R^2
Model:	OLS	Adj. R-squared:	$\bar{R}^2$
Method:	Least Squares	F-statistic:	F_{test}
Date:	Date	Prob (F-statistic):	$\Pr(F_{p,n-p} > F_{\text{test}})$
Time:	Time	Log-Likelihood:	$\ln(\mathcal{L})$
No. Observations:	n	AIC:	AIC
Df Residuals:	$n - p - 1$	BIC:	BIC
Df Model:	p		
Covariance Type:	nonrobust		

Table 1: OLS Regression Results

	coef	std err	t	$p > t $	[0.025, 0.975]
x_0	$\hat{\beta}_0$	$\text{SE}(\hat{\beta}_0)$	$t_{\text{test}}(\hat{\beta}_0)$	$\Pr(t_{\text{df}_{\text{error}}} > t_{\text{test}}(\hat{\beta}_0))$	$\hat{\beta}_0 \pm t_{\frac{\alpha}{2}, \text{test}}(\hat{\beta}_0) \text{SE}(\hat{\beta}_0)$
x_1	$\hat{\beta}_1$	$\text{SE}(\hat{\beta}_1)$	$t_{\text{test}}(\hat{\beta}_1)$	$\Pr(t_{\text{df}_{\text{error}}} > t_{\text{test}}(\hat{\beta}_1))$	$\hat{\beta}_1 \pm t_{\frac{\alpha}{2}, \text{test}}(\hat{\beta}_1) \text{SE}(\hat{\beta}_1)$
$\vdots$					
x_p	$\hat{\beta}_p$	$\text{SE}(\hat{\beta}_p)$	$t_{\text{test}}(\hat{\beta}_p)$	$\Pr(t_{\text{df}_{\text{error}}} > t_{\text{test}}(\hat{\beta}_p))$	$\hat{\beta}_p \pm t_{\frac{\alpha}{2}, \text{test}}(\hat{\beta}_p) \text{SE}(\hat{\beta}_p)$

Table 2: OLS Regression Coefficients

The following quantities will be used in the following discussion:

- Total sum of squares (SST): Total variation in the response variable

$$\text{SST} = \sum_{i=1}^n (y^{(i)} - \bar{y})^2$$

- Error sum of squares (SSE): Total variation in the response variable not explained by the model

$$\text{SSE} = \sum_{i=1}^n (y^{(i)} - \hat{y}(x^{(i)}))^2$$

- Regression sum of squares (SSR): Variation in the response variable explained by the model

$$\text{SSR} = \sum_{i=1}^n (\hat{y}(x^{(i)}) - \bar{y})^2$$

The following relationship holds between the sum of squares:

$$\text{SST} = \text{SSE} + \text{SSR}.$$

The quantities in the regression results are described below:

- The **coefficient of determination** R^2 measures the proportion of the variance in the dependent variable explained by the predictors.

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = \frac{\text{SSR}}{\text{SST}}$$

- The **adjusted coefficient of determination** $\bar{R}^2$ takes into account the number of predictors in the model.

$$\bar{R}^2 = 1 - \frac{\text{SSE}/\text{df}_{\text{error}}}{\text{SST}/\text{df}_{\text{total}}}$$

where $\text{df}_{\text{error}} = n - p - 1$ and $\text{df}_{\text{total}} = n - 1$ are the degrees of freedom for the error and total sum of squares, respectively.

- The **F-statistic** is used to test the overall significance of the model. It is given by:

$$F_{\text{test}} = \frac{(\text{SSR}/p)}{(\text{SSE}/\text{df}_{\text{error}})}$$

- The probability $\Pr(F_{p,n-p} > F_{\text{test}})$ is the probability of observing an F-statistic greater than F_{test} under the null hypothesis that the model is statistically insignificant. A value less than 0.05 suggests that the model is statistically significant.
- The **Log Likelihood** $\ln(\mathcal{L})$ is a measure of how well the model fits the data.

$$\ln(\mathcal{L}) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log\left(\frac{\text{SSE}}{n}\right) - \frac{n}{2}$$

where $\mathcal{L}$ is the maximum value of the likelihood function for the model.

- The **Akaike Information Criterion** (AIC) and **Bayesian Information Criterion** (BIC) are used to compare models. They are given by:

$$\begin{aligned} \text{AIC} &= 2(p+1) - 2\ln(\mathcal{L}) \\ \text{BIC} &= (p+1)\ln(n) - 2\ln(\mathcal{L}) \end{aligned}$$

Smaller values of AIC and BIC suggest a better model fit.

The quantities in the coefficient table are described below:

- The **coefficient** β_i is the estimated value of the parameter β_i .
- The **standard error** $\text{SE}(\hat{\beta})$ is the standard deviation of the sampling distribution of the coefficient. It can be computed as:

$$\text{SE}(\hat{\beta}) = \sqrt{\frac{\text{SSE}}{\text{df}_{\text{error}}} \text{diag}((\mathbf{X}^\top \mathbf{X})^{-1})}$$

The results is a $p+1$ column vector of standard errors for each coefficient.

- The **t-statistic** $t_{\text{test}}(\hat{\beta})$ is used to test the null hypothesis that the coefficient is equal to 0. It is given by:

$$t_{\text{test}}(\hat{\beta}) = \frac{\hat{\beta}}{\text{SE}(\hat{\beta})}$$

- The **p-value** $p > |t|$ is the probability of observing a t-statistic greater than $|t|$ under the null hypothesis that the coefficient is equal to 0. A value less than 0.05 suggests that the coefficient is statistically significant.

$$\Pr(t_{\text{df}_{\text{error}}} > |t_{\text{test}}(\hat{\beta})|) = 2 \Pr(t_{\text{df}_{\text{error}}} > t_{\text{test}}(\hat{\beta}))$$

- The **confidence interval** $[0.025, 0.975]$ is the range of values that the coefficient is likely to lie in. It is given by:

$$[0.025, 0.975] = \hat{\beta} \pm t_{\frac{\alpha}{2}, \text{df}_{\text{error}}} \text{SE}(\hat{\beta})$$

where $t_{\frac{\alpha}{2}, \text{df}_{\text{error}}}$ is the critical value of the t -distribution for a given significance level α . In this case, $\alpha = 0.05$.

Another measure of error of a model is the **root mean squared error** (RMSE), which is given by:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}(x^{(i)}))^2} = \sqrt{\frac{\text{SSE}}{n}}$$

2.8.1 p-Values

The p -values obtained from the F-statistic and t-statistic are used to test the significance of the model and the coefficients, respectively. The p -value is the probability of observing a statistic greater than the observed value under the null hypothesis. A small p -value suggests that the null hypothesis is unlikely to be true, and therefore, the coefficient is statistically significant.

It is important to note that p -values can be misleading when model assumptions are violated.

3 Regularisation

Regularisation is a technique used to prevent overfitting in a model. Including additional predictors or higher order terms in a model may increase the accuracy of the model on the training set, but will often result in poor performance on the test set. In such cases, the model will tend to fit noise in the training set, rather than the underlying relationship between the predictors and the response.

Very complex models are also difficult to inspect or tune due to their size, and often contain many redundant predictors. To improve this model, we must use a validation set to tune the model such that it generalises well to new unseen data.

3.1 Bias and Variance

Bias and variance are two factors that contribute to the total error of a model.

- The **bias** of a model is the error introduced by erroneous assumptions in the model. A model with **high bias** will *underfit* the training data. Increasing the complexity of a model will reduce bias.
- The **variance** of a model is the error introduced by the model's sensitivity to fluctuations in the training set. A model with **high variance** will *overfit* the training data. Reducing the complexity of a model will reduce variance.

The goal of regularisation is to find a model that minimises both bias and variance. This is known as the **bias-variance trade-off**. To formally state the bias-variance trade-off, consider a dataset which was generated by

$$y^{(i)} = y^*(x^{(i)}) + \xi^{(i)}$$

where $y^*(x^{(i)})$ is some underlying function of the predictors, and $\xi^{(i)}$ is observation noise with distribution $\xi \sim N(0, \sigma^2)$. Our goal is to predict the underlying function $y^*(x)$ from the observations $(x^{(i)}, y^{(i)})$.

If we take a test example (x, y) such that $y = y^*(x) + \xi$, and measure the expected test error (averaged over the training set and randomness of ξ), we find that:

$$\text{MSE}(x) = \mathbb{E} \left[(y - \hat{y}(x))^2 \right].$$

Assuming independence between y^* and the noise ξ , we can decompose the expected test error into

three components:

$$\begin{aligned}
 \text{MSE}(x) &= \mathbb{E} \left[(y - \hat{y}(x))^2 \right] \\
 &= \mathbb{E} \left[(\xi + (y^*(x) - \hat{y}(x)))^2 \right] \\
 &= \mathbb{E} [\xi^2] + \mathbb{E} \left[(y^*(x) - \hat{y}(x))^2 \right] \\
 &= \sigma^2 + \mathbb{E} \left[(y^*(x) - \hat{y}(x))^2 \right]
 \end{aligned}$$

Let us define $\hat{y}_{\text{avg}}(x) = \mathbb{E} [\hat{y}(x)]$ as the average model, obtained by drawing an infinite number of datasets, training on them, and averaging their predictions on x . It can be shown that $\hat{y}_{\text{avg}}(x)$ is approximately equal the model obtained by training on a *single* dataset with an infinite number of samples. Let us introduce the constant $c = \hat{y}_{\text{avg}}(x) - y^*(x)$, so that:

$$\begin{aligned}
 \text{MSE}(x) &= \sigma^2 + \mathbb{E} \left[(y^*(x) - \hat{y}(x))^2 \right] \\
 &= \sigma^2 + \mathbb{E} \left[((y^*(x) - \hat{y}_{\text{avg}}(x)) + (\hat{y}_{\text{avg}}(x) - \hat{y}(x)))^2 \right] \\
 &= \sigma^2 + \mathbb{E} \left[(\hat{y}_{\text{avg}}(x) - y^*(x))^2 \right] + \mathbb{E} \left[(\hat{y}(x) - \hat{y}_{\text{avg}}(x))^2 \right] \\
 &= \sigma^2 + (\hat{y}_{\text{avg}}(x) - y^*(x))^2 + \mathbb{E} \left[(\hat{y}(x) - \hat{y}_{\text{avg}}(x))^2 \right] \\
 &= \underbrace{\sigma^2}_{\text{Irreducible Error}} + \underbrace{(\hat{y}_{\text{avg}}(x) - y^*(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E} \left[(\hat{y}(x) - \hat{y}_{\text{avg}}(x))^2 \right]}_{\text{Variance}}.
 \end{aligned}$$

The constant c is the **bias** of $\hat{y}(x)$, relative to the average model $y^*(x)$. Regularisation therefore aims to minimise the total error of a model.

3.2 Regularisation Techniques

Regularisation adds a penalty term to the cost function of a model to penalise large coefficients, reducing the complexity and variance of a model. Two common regularisation techniques are:

- L1 LASSO Regularisation
- L2 Ridge Regularisation

The new regularised cost function is given by:

$$J_{\lambda}(\beta) = J(\beta) + \lambda R(\beta)$$

where $J(\beta)$ is the original cost function, $\lambda \geq 0$ is the regularisation parameter that dictates the strength of the regularisation, and $R(\beta)$ is the regulariser. The regulariser has the form

$$R(\beta) = \|\beta\|_l^l = \frac{1}{l} \sum_{i=1}^p |\beta_i|^l$$

where $\|\cdot\|_l$ represents a L^p -norm, denoted l here to avoid confusion with the number of predictors p . Note that the regulariser is applied to all coefficients except the intercept term β_0 .

3.3 LASSO Regularisation

LASSO (Least Absolute Shrinkage and Selection Operator) regularisation is a regularisation technique used for feature selection as it can shrink coefficients to 0. The regulariser penalises the absolute size of each coefficient through the L^1 -norm:

$$R_{\text{LASSO}}(\boldsymbol{\beta}) = \|\boldsymbol{\beta}\|_1 = \sum_{i=1}^p |\beta_i| = \mathbf{1}^\top |\boldsymbol{\beta}|$$

where $\mathbf{1}$ is a p -dimensional vector of ones and $|\boldsymbol{\beta}|$ is the element-wise absolute value of $\boldsymbol{\beta}$. The regularised cost function is therefore given by:

$$\begin{aligned} J_\lambda(\boldsymbol{\beta}) &= J(\boldsymbol{\beta}) + \lambda R_{\text{LASSO}}(\boldsymbol{\beta}) \\ &= \frac{1}{2n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \|\boldsymbol{\beta}\|_1. \end{aligned}$$

Attempting to use the same derivation as in the previous section reveals that it is no longer possible to analytically solve for the optimal parameters $\boldsymbol{\beta}$. This is shown below:

$$\begin{aligned} \frac{\partial J_\lambda(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} &= \frac{\partial J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} + \lambda \frac{\partial R_{\text{LASSO}}(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} \\ &= \frac{1}{n} (\mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} - \mathbf{X}^\top \mathbf{y}) + \lambda \text{sgn}(\boldsymbol{\beta}) \end{aligned}$$

If we set this equal to 0, we find an implicit relationship between $\boldsymbol{\beta}$ and λ :

$$\begin{aligned} \frac{\partial J_\lambda(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} &= \mathbf{0} \\ \mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} - \mathbf{X}^\top \mathbf{y} + n \lambda \text{sgn}(\boldsymbol{\beta}) &= \mathbf{0} \\ \mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} &= \mathbf{X}^\top \mathbf{y} - n \lambda \text{sgn}(\boldsymbol{\beta}) \\ \boldsymbol{\beta} &= (\mathbf{X}^\top \mathbf{X})^{-1} (\mathbf{X}^\top \mathbf{y} - n \lambda \text{sgn}(\boldsymbol{\beta})). \end{aligned}$$

This implicit relationship must be solved using an iterative method.

3.4 Ridge Regularisation

Ridge regularisation is a regularisation technique that is used to reduce large coefficients. This is done using the L^2 -norm, which penalises the squared size of each coefficient:

$$R_{\text{Ridge}}(\boldsymbol{\beta}) = \frac{1}{2} \|\boldsymbol{\beta}\|_2^2 = \frac{1}{2} \sum_{i=1}^p \beta_i^2 = \frac{1}{2} \boldsymbol{\beta}^\top \boldsymbol{\beta}$$

The regularised cost function for this technique becomes:

$$\begin{aligned} J_\lambda(\boldsymbol{\beta}) &= J(\boldsymbol{\beta}) + \lambda R_{\text{Ridge}}(\boldsymbol{\beta}) \\ &= \frac{1}{2n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2 + \lambda \frac{1}{2} \|\boldsymbol{\beta}\|_2^2. \end{aligned}$$

Unlike LASSO regularisation, it is possible to analytically solve for the optimal parameters β using the following derivation:

$$\begin{aligned}\frac{\partial J_{\lambda}(\beta)}{\partial \beta} &= \frac{\partial J(\beta)}{\partial \beta} + \lambda \frac{\partial R_{\text{Ridge}}(\beta)}{\partial \beta} \\ &= \frac{1}{n} (\mathbf{X}^{\top} \mathbf{X} \beta - \mathbf{X}^{\top} \mathbf{y}) + \lambda \beta\end{aligned}$$

Setting this equal to 0 we find:

$$\begin{aligned}\frac{\partial J_{\lambda}(\beta)}{\partial \beta} &= \mathbf{0} \\ \mathbf{X}^{\top} \mathbf{X} \beta - \mathbf{X}^{\top} \mathbf{y} + n \lambda \beta &= \mathbf{0} \\ \mathbf{X}^{\top} \mathbf{X} \beta + n \lambda \beta &= \mathbf{X}^{\top} \mathbf{y} \\ \mathbf{X}^{\top} \mathbf{X} \beta + n \lambda \beta &= \mathbf{X}^{\top} \mathbf{y} \\ (\mathbf{X}^{\top} \mathbf{X} + n \lambda \mathbf{I}) \beta &= \mathbf{X}^{\top} \mathbf{y} \\ \beta &= (\mathbf{X}^{\top} \mathbf{X} + n \lambda \mathbf{I})^{-1} \mathbf{X}^{\top} \mathbf{y}.\end{aligned}$$

The term “ridge” refers to the ill-conditioned nature of the matrix inverse performed on the covariance matrix $\mathbf{X}^{\top} \mathbf{X}$ due to multicollinearity. Ridge regularisation aims to smooth the cost function near the ridge, by adding a penalty term to the diagonal of the covariance matrix. This reduces the likelihood of the matrix being singular when it’s columns are linearly dependent.

3.5 Ridge vs LASSO Regularisation

When choosing between ridge and LASSO regularisation, it is important to understand their advantages and disadvantages.

- Ridge regression is typically faster to train than LASSO regression, as it has a closed-form solution.
- LASSO regression produces a simpler model than ridge regression, as it can shrink coefficients to 0, effectively performing feature selection.
- LASSO regression performs faster during the testing phase, as the model has fewer coefficients to compute.

To understand why LASSO regression can eliminate coefficients, consider the optimisation problem where the objective function is $J(\beta)$, with the constraint $R(\beta) \leq t$:

$$\min_{\beta} J(\beta) \quad \text{subject to} \quad R(\beta) \leq t$$

where t is a constant that is inversely proportional to λ . If we imagine a problem where $p = 2$, and plot the level curves of J on the feature plane, we can see that the constraints for LASSO and ridge regularisation have different characteristics.

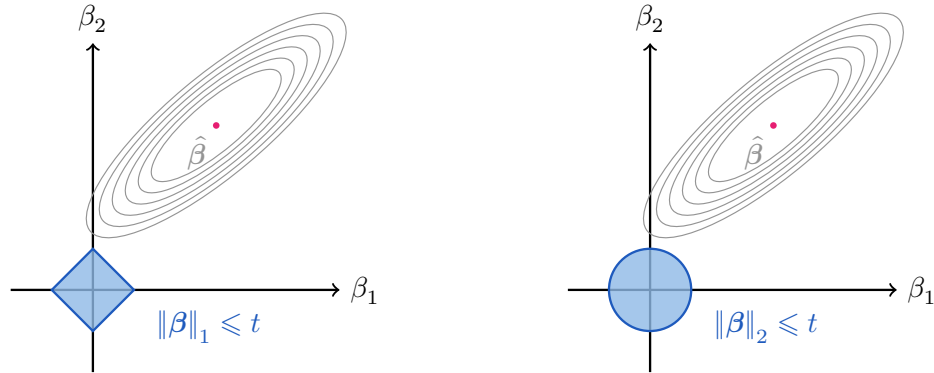


Figure 2: Comparison of constrain region $R \leq t$ for LASSO (L1) and ridge (L2) regression.

The sharp corners of the LASSO constraint region cause the level curves of J to intersect the constraint region at the axes, leading to a sparse solution. Because of its constant radius, ridge regression tends to produce a solution with small coefficients that are not necessarily zero.

4 Classification

Classification is a supervised learning task that predicts classes of categorical data. There are two main types of classification:

- **Binary classification:** The response variable has two classes.
- **Multiclass classification:** The response variable has K classes.

There are several algorithms that can be used to solve these classification problems, and we will discuss a few in the following sections.

4.1 Logistic Regression

Logistic regression is a classification algorithm that models the probability of a binary response variable. The model aims to predict the probability that a given observation belongs to a particular class by producing an output between 0 and 1. The model considers the **logistic** function (or **sigmoid** function) defined:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z}$$

where we use the variable z to represent a linear combination of the predictors:

$$z = \sum_{i=1}^n \beta_i \mathbf{x} = \mathbf{w}^\top \mathbf{x} + b$$

where $\mathbf{w} = [\beta_1, \dots, \beta_n]$ is a vector of **weights**, and $b = \beta_0$ is the **bias** term. The model is then defined as:

$$\hat{y} = \sigma(z) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

where we assume that the probability of y being equal to 1 is given by:

$$\Pr(y = 1 \mid \mathbf{x}; \beta) = \hat{y},$$

so that the probability of y being equal to 0 is given by:

$$\Pr(y = 0 \mid \mathbf{x}; \beta) = 1 - \hat{y},$$

both conditioned on the predictors $\mathbf{x}$. If we apply the method of maximum likelihood estimation to this model, we can derive a loss function for logistic regression. First let us construct a general expression for the probability of y given $\mathbf{x}$:

$$p(y^{(i)} \mid \mathbf{x}^{(i)}; \beta) = \hat{y}(\mathbf{x}^{(i)})^{y^{(i)}} (1 - \hat{y}(\mathbf{x}^{(i)}))^{1-y^{(i)}}$$

so that we can define the likelihood function as:

$$\mathcal{L}(\beta) = \prod_{i=1}^n p(y^{(i)} \mid \mathbf{x}^{(i)}; \beta) = \prod_{i=1}^n \hat{y}(\mathbf{x}^{(i)})^{y^{(i)}} (1 - \hat{y}(\mathbf{x}^{(i)}))^{1-y^{(i)}}.$$

Then let us take the logarithm of the likelihood function:

$$\begin{aligned} \ell(\beta) &= \ln \mathcal{L}(\beta) \\ &= \ln \left[\prod_{i=1}^n \hat{y}(\mathbf{x}^{(i)})^{y^{(i)}} (1 - \hat{y}(\mathbf{x}^{(i)}))^{1-y^{(i)}} \right] \\ &= \sum_{i=1}^n \ln \left[\hat{y}(\mathbf{x}^{(i)})^{y^{(i)}} (1 - \hat{y}(\mathbf{x}^{(i)}))^{1-y^{(i)}} \right] \\ &= \sum_{i=1}^n \ln \left[\hat{y}(\mathbf{x}^{(i)})^{y^{(i)}} \right] + \sum_{i=1}^n \ln \left[(1 - \hat{y}(\mathbf{x}^{(i)}))^{1-y^{(i)}} \right] \\ &= \sum_{i=1}^n [y^{(i)} \ln \hat{y}(\mathbf{x}^{(i)}) + (1 - y^{(i)}) \ln (1 - \hat{y}(\mathbf{x}^{(i)}))]. \end{aligned}$$

Therefore, to maximise the log-likelihood function, we will minimise negative log-likelihood function:

$$L(\beta) = - \left[y \ln \hat{y}(\mathbf{x}) + (1 - y) \ln (1 - \hat{y}(\mathbf{x})) \right]$$

which we will define as the loss function for logistic regression.

4.1.1 Multiclass Logistic Regression

If our classification problem is multiclass, our model must be able to predict the probability that an observation belongs to class k . Given $K \geq 2$ classes, let us try to generalise the sigmoid function we used for binary classification. Before we can do so, we must note the following:

1. Each class will have its own set of weights and biases.
2. The sum of the probabilities of all classes predicted by the model must be equal to 1.

3. The model only has $K-1$ independent probabilities, and therefore $K-1$ specifiable coefficients β_k , because the probability of the last class can be defined as the complement of the other classes.

Let us assume that the model has the form:

$$\Pr(y = k \mid x; \beta) = \frac{e^{\beta_k^\top \mathbf{x}}}{e^{\beta_1^\top \mathbf{x}} + \dots + e^{\beta_K^\top \mathbf{x}}}, \quad k = 1, \dots, K-1$$

where each class k has a probability $e^{\beta_k^\top \mathbf{x}}$ parametrised by coefficients β_k , and the denominator ensures that the sum of the probabilities is equal to 1. Now observe that we have $K-1$ equations for K coefficient vectors β_1 through β_K :

$$\begin{aligned} \Pr(y = 1 \mid x; \beta) &= \frac{e^{\beta_1^\top \mathbf{x}}}{e^{\beta_1^\top \mathbf{x}} + \dots + e^{\beta_K^\top \mathbf{x}}} \\ \Pr(y = 2 \mid x; \beta) &= \frac{e^{\beta_2^\top \mathbf{x}}}{e^{\beta_1^\top \mathbf{x}} + \dots + e^{\beta_K^\top \mathbf{x}}} \\ &\vdots \\ \Pr(y = K-1 \mid x; \beta) &= \frac{e^{\beta_{K-1}^\top \mathbf{x}}}{e^{\beta_1^\top \mathbf{x}} + \dots + e^{\beta_K^\top \mathbf{x}}} \end{aligned}$$

as the probability of observing the K th class can be defined using the complement of the other classes:

$$\Pr(y = K \mid x; \beta) = 1 - \sum_{k=1}^{K-1} \Pr(y = k \mid x; \beta).$$

Hence, we have an underdetermined system of equations, where β_K must be a free variable. If we choose to set this variable to 0, we find a unique solution for the coefficients β_1 through β_{K-1} :

$$\Pr(y = k \mid x; \beta) = \frac{e^{\beta_k^\top \mathbf{x}}}{1 + \sum_{k=1}^{K-1} e^{\beta_k^\top \mathbf{x}}}, \quad k = 1, \dots, K-1.$$

This model is therefore a generalisation of the model obtained using the sigmoid function to multiple classes, where we now use the **softmax** function. Using a similar derivation to the one we used for logistic regression, we can formulate the maximum likelihood estimator for a multiclass logistic regression model:

$$\begin{aligned} \mathcal{L}(\beta) &= \Pr(y^{(i)} \mid \mathbf{X}^{(i)}; \beta) \\ \mathcal{L}(\beta) &= \prod_{i=1}^n \Pr(y^{(i)} \mid \mathbf{x}^{(i)}; \beta) \\ \ln(\mathcal{L}(\beta)) &= \ln \prod_{i=1}^n \Pr(y^{(i)} \mid \mathbf{x}^{(i)}; \beta) \\ \ell(\beta) &= \sum_{i=1}^n \ln \Pr(y^{(i)} \mid \mathbf{x}^{(i)}; \beta) \\ &= \sum_{i=1}^n \ln \left[\frac{e^{\beta_{y^{(i)}}^\top \mathbf{x}^{(i)}}}{1 + \sum_{k=1}^{K-1} e^{\beta_k^\top \mathbf{x}^{(i)}}} \right]. \end{aligned}$$

Therefore, to maximise the log-likelihood function, we will minimise the negative log-likelihood function:

$$L(\boldsymbol{\beta}) = -\ln \left[\frac{e^{\boldsymbol{\beta}_y^\top \mathbf{x}}}{1 + \sum_{k=1}^{K-1} e^{\boldsymbol{\beta}_k^\top \mathbf{x}}} \right].$$

When predictions lie between 0 and 1, as they do in logistic regression, this loss function is referred to as the **cross-entropy** loss function. This applies to both the binary and multiclass logistic regression models.

4.2 Support Vector Machines

Support Vector Machines (SVM) are a supervised learning model used for binary classification. The model works by finding a hyperplane that separates classes with the maximum margin. Consider a dataset of n points of the form:

$$\{(\mathbf{x}^{(i)}, y^{(i)})\}$$

where $\mathbf{x}^{(i)} \in \mathbb{R}^p$ is a p -dimensional and $y^{(i)} \in \{-1, +1\}$ indicates the class of the i th point. We can model a p -dimensional hyperplane by the equation:

$$\mathbf{w}^\top \mathbf{x} - b = 0$$

where $\mathbf{w} \in \mathbb{R}^p$ is a vector normal to the hyperplane and $b \in \mathbb{R}$ is the bias term. In this model, the distance between the hyperplane and the origin is given by $|b|/\|\mathbf{w}\|$.

4.2.1 Hard Margin SVM

If a dataset is linearly separable, we can select two parallel hyperplanes that separate the classes, such that the distance between them is maximised. We start by defining the **functional margin** of a point $\mathbf{x}^{(i)}$ as

$$\hat{\gamma}^{(i)} = y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} - b)$$

and the **functional margin** of the optimal hyperplane as

$$\hat{\gamma} = \min_{i=1, \dots, n} \hat{\gamma}^{(i)}$$

This value represents the **correctness** and **confidence** of the classification of a point for a given hyperplane:

- A hyperplane correctly classifies a point when $\hat{\gamma}^{(i)} > 0$.
- A hyperplane confidently classifies a point when $|\hat{\gamma}^{(i)}| \gg 0$.

Using this definition, we also define the **geometric margin** of a point $\mathbf{x}^{(i)}$, to allow us to compare the confidence of points across different hyperplanes:

$$\gamma^{(i)} = \frac{\hat{\gamma}^{(i)}}{\|\mathbf{w}\|}.$$

In a similar manner, the **geometric margin** of the optimal hyperplane is defined:

$$\gamma = \frac{\hat{\gamma}}{\|\mathbf{w}\|}$$

To understand why we define the functional margin in this way, consider the points $\mathbf{x}_+$ that are classified as +1, and the points $\mathbf{x}_-$ that are classified as -1. We wish to constrain the two classes by the hyperplanes:

$$\begin{aligned}\mathbf{w}^\top \mathbf{x}_+ - b &\geq +\hat{\gamma} \\ \mathbf{w}^\top \mathbf{x}_- - b &\leq -\hat{\gamma}\end{aligned}$$

If we multiply both equations by y , which for $\mathbf{x}_+$ is +1, and for $\mathbf{x}_-$ is -1, we find:

$$\begin{aligned}\begin{cases} y(\mathbf{w}^\top \mathbf{x}_+ - b) \geq \hat{\gamma} \\ y(\mathbf{w}^\top \mathbf{x}_- - b) \geq \hat{\gamma} \end{cases} \\ \therefore y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} - b) \geq \hat{\gamma}.\end{aligned}$$

We can then pose the following optimisation problem:

$$\begin{aligned}\max_{\hat{\gamma}, \mathbf{w}, b} \quad & \frac{\hat{\gamma}}{\|\mathbf{w}\|} \\ \text{subject to} \quad & y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} - b) \geq \hat{\gamma}, \quad i = 1, \dots, n\end{aligned}$$

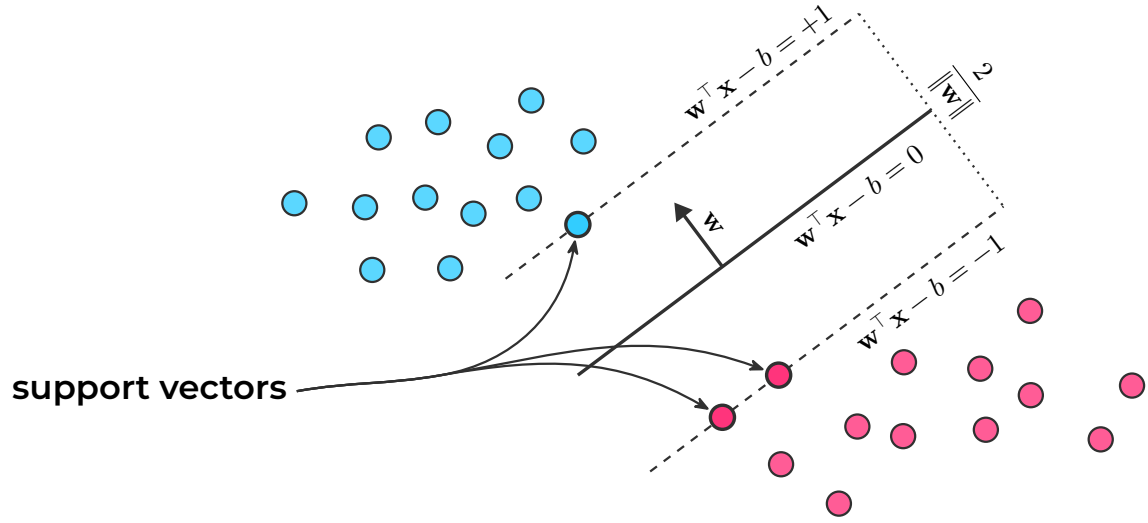
This is a non-convex problem, as the objective function contains the nonlinear term $\|\mathbf{w}\|$. If we impose the constraint

$$\hat{\gamma} = 1$$

and note that maximising $\hat{\gamma}/\|\mathbf{w}\| = 1/\|\mathbf{w}\|$ is the same as minimising $\|\mathbf{w}\|^2$, we can solve the equivalent problem:

$$\begin{aligned}\min_{\mathbf{w}, b} \quad & \frac{1}{2}\|\mathbf{w}\|^2 \\ \text{subject to} \quad & y^{(i)}(\mathbf{w}^\top \mathbf{x}^{(i)} - b) \geq 1, \quad i = 1, \dots, n.\end{aligned}$$

This is an optimisation problem with a convex quadratic objective function and linear constraints whose solution gives us the **optimal margin classifier**. The following figure shows a geometric interpretation of the optimal margin classifier:



The **decision boundary** is completely determined by the points that lie nearest to it. These points are called **support vectors**.

4.2.2 The Dual Problem

We can reformulate the optimisation problem for the optimal margin classifier using **Lagrange multipliers**.

$$\begin{aligned}\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) &= \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n \alpha^{(i)} (1 - y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} - b)) \\ &= \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n \alpha^{(i)} (1 - y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)} + y^{(i)} b)\end{aligned}$$

where $\boldsymbol{\alpha} = (\alpha^{(1)}, \dots, \alpha^{(n)})$ is a vector of Lagrange multipliers. The **dual problem** is then given by:

$$\begin{aligned}& \max_{\boldsymbol{\alpha}} \quad \min_{\mathbf{w}, b} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) \\ & \text{subject to} \quad \alpha^{(i)} \geq 0, \quad i = 1, \dots, n \\ & \quad \quad \quad \frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})}{\partial \mathbf{w}} = 0 \\ & \quad \quad \quad \frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})}{\partial b} = 0.\end{aligned}$$

The two partial derivatives are given by:

$$\begin{aligned}\frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})}{\partial \mathbf{w}} &= \mathbf{w} - \sum_{i=1}^n \alpha^{(i)} y^{(i)} \mathbf{x}^{(i)} = \mathbf{0} \quad \implies \quad \mathbf{w} = \sum_{i=1}^n \alpha^{(i)} y^{(i)} \mathbf{x}^{(i)} \\ \frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})}{\partial b} &= \sum_{i=1}^n \alpha^{(i)} y^{(i)} = 0.\end{aligned}$$

The Lagrangian then simplifies to:

$$\begin{aligned}\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) &= \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n \alpha^{(i)} (1 - y^{(i)} \mathbf{w}^\top \mathbf{x}^{(i)} + y^{(i)} b) \\ &= \frac{1}{2} \left(\sum_{i=1}^n \alpha^{(i)} y^{(i)} \mathbf{x}^{(i)} \right)^\top \left(\sum_{i=1}^n \alpha^{(i)} y^{(i)} \mathbf{x}^{(i)} \right) \\ &\quad + \sum_{i=1}^n \alpha^{(i)} - \sum_{i=1}^n \sum_{j=1}^n \alpha^{(i)} \alpha^{(j)} y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle + b \sum_{i=1}^n \alpha^{(i)} y^{(i)} \\ &= \sum_{i=1}^n \alpha^{(i)} - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha^{(i)} \alpha^{(j)} y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\ &= \boldsymbol{\alpha}^\top \mathbf{1} - \frac{1}{2} \|(\boldsymbol{\alpha} \odot \mathbf{y})^\top \mathbf{X}\|^2\end{aligned}$$

where $\mathbf{1}$ is a vector of ones, and $\odot$ denotes the element-wise product. Therefore, in this form, we find that our problem only depends on the inner product of points in the dataset:

$$\begin{aligned}\max_{\boldsymbol{\alpha}} \quad & \sum_{i=1}^n \alpha^{(i)} - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha^{(i)} \alpha^{(j)} y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\ \text{subject to} \quad & \alpha^{(i)} \geq 0, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha^{(i)} y^{(i)} = 0.\end{aligned}$$

By examining the dual form of the optimisation problem, we can see that our algorithm is in terms of inner products between input vectors. This will be important when we consider kernel methods in the next section.

4.2.3 Kernel Methods

Consider a dataset where a linear model is not sufficient. We can transform the input vectors $\mathbf{x}$ into a higher-dimensional space using the **feature map** $\phi : \mathbb{R}^p \rightarrow \mathbb{R}^d$ such that

$$\mathbf{x} \mapsto \phi(\mathbf{x}).$$

Let us define a **kernel function** corresponding to the feature map ϕ as a function $K : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$ which satisfies:

$$K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(j)}) \rangle_V.$$

K is a symmetric positive semi-definite kernel because it satisfies Mercer's theorem, that is,

$$\sum_{i=1}^p \sum_{j=1}^p c_i c_j K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \geq 0.$$

Alternatively, we can construct the Gram matrix $\mathbf{K}$ where $\mathbf{K}_{ij} = K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, demonstrating that $\mathbf{K}$ is positive semi-definite.

Our goal is then to find a method to compute the inner product $\langle \phi(\mathbf{x}^{(i)}), \phi(\mathbf{x}^{(j)}) \rangle_V$ without explicitly computing ϕ , so that we can replace all inner products involving the input vector with the kernel function K . This allows us to work with ϕ , which can be infinite in dimension, without knowing its explicit form.

This is the idea behind the **kernel trick**, which allows us to solve the dual form of the optimisation problem using a kernel function.

4.2.4 Common Kernels

In this section, we will use the symbols $\mathbf{x}$ and $\mathbf{z}$ to represent input vectors $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(j)}$ for brevity.

Consider the function $K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$. Through some algebraic manipulation, we can show that this is a valid kernel function:

$$\begin{aligned} K(\mathbf{x}, \mathbf{z}) &= \mathbf{x}^\top \mathbf{z} \\ &= \langle \mathbf{x}, \mathbf{z} \rangle_V. \end{aligned}$$

This forms our first trivial kernel function, the **linear kernel**, where $\phi(\mathbf{x}) = \mathbf{x}$. Now consider the kernel function $K(\mathbf{x}, \mathbf{z}) = f(\mathbf{x}) f(\mathbf{z})$, for some function $f: \mathbb{R}^p \rightarrow \mathbb{R}$. We can show that this is also a valid kernel function by letting $\phi(\mathbf{x}) = f(\mathbf{x})$:

$$\begin{aligned} K(\mathbf{x}, \mathbf{z}) &= f(\mathbf{x}) f(\mathbf{z}) \\ &= \langle f(\mathbf{x}), f(\mathbf{z}) \rangle_V. \end{aligned}$$

Let K_1 and K_2 be valid kernel functions, and let $\alpha_1, \alpha_2 \geq 0$, then, the following transformations also produce valid kernel functions:

- Scalar multiplication: $K(\mathbf{x}, \mathbf{z}) = \alpha K_1(\mathbf{x}, \mathbf{z})$.
- Adding a constant: $K(\mathbf{x}, \mathbf{z}) = K_1(\mathbf{x}, \mathbf{z}) + \alpha$.
- Linear combination: $K(\mathbf{x}, \mathbf{z}) = \alpha_1 K_1(\mathbf{x}, \mathbf{z}) + \alpha_2 K_2(\mathbf{x}, \mathbf{z})$.
- Element-wise product: $K(\mathbf{x}, \mathbf{z}) = K_1(\mathbf{x}, \mathbf{z}) K_2(\mathbf{x}, \mathbf{z})$.
- Polynomial map: $K(\mathbf{x}, \mathbf{z}) = P_d(K_1(\mathbf{x}, \mathbf{z}))$ where P_d is a polynomial of degree d with positive coefficients.
- Exponential map: $K(\mathbf{x}, \mathbf{z}) = \exp(K_1(\mathbf{x}, \mathbf{z}))$.

These transformations allow us to construct a wide variety of kernel functions that satisfy the conditions defined above. Some common kernel functions are:

- Polynomial kernel: $K(\mathbf{x}, \mathbf{z}) = (\gamma \mathbf{x}^\top \mathbf{z} + r)^d$ for $d \in \mathbb{N}$.

- Gaussian kernel: $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$.
- Sigmoid kernel: $K(\mathbf{x}, \mathbf{z}) = \tanh(\gamma \mathbf{x}^\top \mathbf{z} + r)$.

where γ controls the influence of each training example on the decision boundary, and r is a bias term.

The choice of kernel is problem dependent, but generally, points that are similar should produce a large value of K , and points that are dissimilar should produce a small value of K .

A good starting point is the linear kernel, which allows us to visualise the decision boundary and understand the problem. If this kernel does not perform well, the Gaussian kernel should be used, as it only has one hyperparameter that needs to be tuned.

- A small value of γ extends influence of a point and leads to a smooth decision boundary.
- A large value of γ results in a more complex decision boundary. This can lead to overfitting.

The polynomial kernel is often more difficult to tune, as the degree of the polynomial can have a large effect on the decision boundary and the sigmoid kernel should only be used in specific cases.

4.2.5 Soft Margin SVM

Our discussion so far has only considered a linearly separable dataset. When a dataset is not linearly separable, the optimal margin classifier algorithm may fail to converge or return a suboptimal solution. This is due to the fact that the optimal margin classifier is sensitive to outliers, and may not generalise well to new data.

To prevent this, we will introduce the **slack variable** $\xi^{(i)}$ for each point $\mathbf{x}^{(i)}$, and reformulate the dual problem as follows:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi^{(i)} \\ \text{subject to} \quad & y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} - b) \geq 1 - \xi^{(i)}, \quad i = 1, \dots, n \\ & \xi^{(i)} \geq 0, \quad i = 1, \dots, n \end{aligned}$$

This modification now allows points to be misclassified with a penalty $C\xi^{(i)}$. C is a regularisation parameter (sometimes called a box constraint) that controls the trade-off between the margin and the slack variables.

- A small value of C will penalise misclassifications less and produce a wider margin.
- A large value of C will penalise misclassifications more and produce a smaller margin.

$C = \infty$ is equivalent to the hard margin SVM, as it forces $\xi^{(i)} = 0$ for all i . Note that an alternative formulation of the soft margin SVM may use the **hinge loss** function instead:

$$\ell(\mathbf{w}, b) = \max(0, 1 - y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} - b)).$$

This function is zero when a point is classified correctly, and increases linearly with the distance from the decision boundary when a point is misclassified. An equivalent formulation of the soft margin SVM is then:

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \left[\frac{1}{n} \sum_{i=1}^n \max(0, 1 - y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} - b)) \right].$$

The dual problem can be solved in a similar manner to the hard margin SVM. The Lagrangian is given by:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi^{(i)} + \sum_{i=1}^n \alpha^{(i)} (1 - y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} - b) - \xi^{(i)}) + \sum_{i=1}^n \beta^{(i)} (-\xi^{(i)}).$$

The partial derivatives for this problem are identical to the hard margin SVM, with the addition of the partial derivative with respect to $\xi^{(i)}$:

$$\frac{\partial \mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta})}{\partial \xi^{(i)}} = C - (\alpha^{(i)} + \beta^{(i)}) = 0 \implies C = \alpha^{(i)} + \beta^{(i)}.$$

This results in the following Lagrangian:

$$\begin{aligned} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\beta}) &= \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi^{(i)} + \sum_{i=1}^n \alpha^{(i)} (1 - y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} - b) - \xi^{(i)}) + \sum_{i=1}^n \beta^{(i)} (-\xi^{(i)}) \\ &= \frac{1}{2} \left(\sum_{i=1}^n \alpha^{(i)} y^{(i)} \mathbf{x}^{(i)} \right)^\top \left(\sum_{i=1}^n \alpha^{(i)} y^{(i)} \mathbf{x}^{(i)} \right) \\ &\quad + \sum_{i=1}^n \alpha^{(i)} - \sum_{i=1}^n \sum_{j=1}^n \alpha^{(i)} \alpha^{(j)} y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle + b \sum_{i=1}^n \alpha^{(i)} y^{(i)} \\ &\quad + C \sum_{i=1}^n \xi^{(i)} - \sum_{i=1}^n \xi^{(i)} (\alpha^{(i)} + \beta^{(i)}) \\ &= \sum_{i=1}^n \alpha^{(i)} - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha^{(i)} \alpha^{(j)} y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\ &= \boldsymbol{\alpha}^\top \mathbf{1} - \frac{1}{2} \|(\boldsymbol{\alpha} \odot \mathbf{y})^\top \mathbf{X}\|^2 \end{aligned}$$

The dual problem is then given by:

$$\begin{aligned} \max_{\boldsymbol{\alpha}, \boldsymbol{\beta}} \quad & \sum_{i=1}^n \alpha^{(i)} - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha^{(i)} \alpha^{(j)} y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\ \text{subject to} \quad & \alpha^{(i)} \geq 0, \quad i = 1, \dots, n \\ & \beta^{(i)} \geq 0, \quad i = 1, \dots, n \\ & C = \alpha^{(i)} + \beta^{(i)}, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha^{(i)} y^{(i)} = 0. \end{aligned}$$

As $\beta^{(i)} = C - \alpha^{(i)}$ does not appear in the objective function, it can be eliminated from the problem:

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & \sum_{i=1}^n \alpha^{(i)} - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha^{(i)} \alpha^{(j)} y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle \\ \text{subject to} \quad & 0 \leq \alpha^{(i)} \leq C, \quad i = 1, \dots, n \\ & \sum_{i=1}^n \alpha^{(i)} y^{(i)} = 0. \end{aligned}$$

4.3 K-Nearest Neighbours

The k -nearest neighbours (k -NN) algorithm is a simple classification algorithm that classifies a point based on the majority class of its k nearest neighbours. The algorithm is non-parametric, meaning that it does not make any assumptions about the underlying distribution of the data. In this algorithm:

- The training stage only consists of storing the feature vectors and their corresponding class labels.
- The testing stage then finds the k nearest neighbours of a given point and assigns it the majority class of those neighbours.

When classifying a point in the test set, we must choose an appropriate number of neighbours k .

- A small value of k may lead to poor performance on noisy data, as it will be sensitive to outliers.
- A large value of k may lead to misclassification when points are close to the decision boundary, or when classes are imbalanced.

The distance metric used to find the nearest neighbour is problem dependent, but the following are common choices:

- **Manhattan distance:** $\|\mathbf{x}^{(i)} - \mathbf{x}\|_1$.
- **Euclidean distance:** $\|\mathbf{x}^{(i)} - \mathbf{x}\|_2$.
- **Minkowski distance:** $\|\mathbf{x}^{(i)} - \mathbf{x}\|_p$ for $p \in \mathbb{N}$.
- **Cosine similarity distance:** $1 - \frac{\mathbf{x}^\top \mathbf{z}}{\|\mathbf{x}\|_2 \|\mathbf{z}\|_2}$ measures the angle between vectors.
- **Mahalanobis distance:** $\sqrt{(\mathbf{x}^{(i)} - \mathbf{x})^\top \mathbf{S}^{-1} (\mathbf{x}^{(i)} - \mathbf{x})}$ assumes $\mathbf{x}$ are distributed according to some distribution, with covariance matrix $\mathbf{S}$.
- **Hamming distance:** Used for binary data, it counts the number of positions at which corresponding symbols are different.

4.4 Random Forest

A random forest is an ensemble learning method that is used for both classification and regression that constructs multiple decision trees at training time to predict an examples class. In classification tasks, the output of the random tree is the class selected by the most trees. It's aim is to produce a diverse set of trees that are uncorrelated, and therefore, when combined, produce a more accurate prediction than any individual tree.

4.4.1 Decision Trees

Decision trees are a classical learning method that iteratively split data based on some criterion. Decision trees consists of two types of nodes:

- **Decision nodes:** Represent a decision that splits data entering that node.
- **End nodes:** Represent a class label with some probability.

A tree is constructed using the entire training set, until one of the following conditions are met:

- The tree reaches a maximum depth.
- The previous decision splits the data into classes that are pure, or nearly pure (i.e., consisting of a single class). This is determined by the **Gini impurity**.
- The information gain from a split is below a certain threshold.

4.4.2 Random Forests

In the random forest algorithm, a number of decision trees are constructed using a random subset of the training data. This can be done in one of two ways:

- **Bootstrap aggregation (bagging):** Sampling with replacement to create a new training set of the same size. This is repeated B times to create B new training sets, for B *different* decision trees.
- **Feature bagging:** Randomly select a subset of features to use at each decision node. This is done to prevent a single feature from dominating the decision.

The depth of the trees is used to control the complexity of the model and should be chosen carefully considering the number of examples in each class, to prevent class domination. Generally, increasing the tree depth also increases the risk of overfitting.

4.5 Multiclass Classification

Each of the above algorithms can be extended to multiclass classification using one of the following methods:

- **One-vs-One:** Pair a class with another class, and train a binary classifier for each pair. This results in $\frac{p(p+1)}{2}$ classifiers.
- **One-vs-All:** Pair a class with all other classes, and train a binary classifier for each group. This results in p classifiers.

It is important to note that the one-vs-all method will suffer from class imbalance when the number of classes is large. In such cases, class weights can be used to counteract class imbalance, where weights are inversely proportional to the amount of data in a class.

4.6 Evaluating a Classification Model

The confusion matrix is a table that is used to describe the performance of a classification model. It has the form:

		Predicted class	
Actual class	Actual Positive	TP True Positives	FN False Negatives
	Actual Negative	FP False Positives	TN True Negatives

The following metrics are used to assess the performance of binary classification models.

Metric	Formula	Interpretation
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Overall performance of model
Precision	$\frac{TP}{TP + FP}$	Accuracy of positive predictions
Recall Sensitivity	$\frac{TP}{TP + FN}$	Coverage of actual positive samples
Sensitivity	$\frac{TN}{TN + FP}$	Coverage of actual negative samples
F1 Score	$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall

Compared to accuracy, precision and recall provide a more detailed analysis of how a model performs. Choosing the right metric depends on what is important for a problem:

- If we wish to minimise false negatives: Use recall.
- If we wish to minimise false positives: Use precision.
- For overall performance: Use accuracy.

These metrics can also be generalised to multiclass classification by computing them for each class. Here we must consider datasets with class imbalance, as the metrics may be biased towards the majority class.

4.7 Model Selection

Deciding on which classification model to use can be difficult and vary from problem to problem. Ideally we want to perform an exhaustive search across a set of hyperparameters for multiple models, and selecting the model that performs best on a test set. This can be done using a grid search over all possible combinations, however this can be computationally expensive. Instead, we may use a random search to search a sparse grid, followed by a more fine-grained search.

Part II

Deep Learning

A neural network is a class of models that is inspired by the human brain. It is made up of interconnected units called **neurons**, that are organised into layers. The upcoming sections will discuss various kinds of neural networks that have been proposed over the past decades.

5 Feedforward Neural Networks

The simplest form of a neural network is the **feedforward neural network**, where data flows from one layer to the next.

- The *first* layer is called the **input layer**, and it receives the input data.
- The *last* layer is called the **output layer**, and it produces the output.
- Any layers in between are called **hidden layers**.

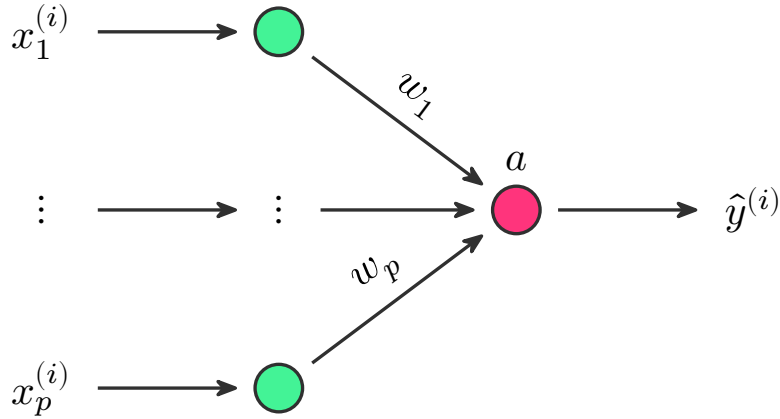
Each neuron in a layer is connected to a neuron in the previous layer via an **edge**. Each edge has an associated **weight** that determines the strength of its connection to the next neuron. The **output** of a neuron is computed by applying a *non-linear* function on the weighted sum of its inputs, called the **activation function**.

5.1 Fully Connected Networks

The earliest forms of neural networks were **fully connected**, meaning that each neuron in a layer is connected to every neuron in the next layer. This is also known as a **dense** layer.

5.1.1 Perceptron

The simplest form of a neural network is the **perceptron**, which is a single-layer neural network with one neuron.



In this model, input nodes are coloured green and the output node is coloured pink. The output of this neuron, for the i -th observation is given by:

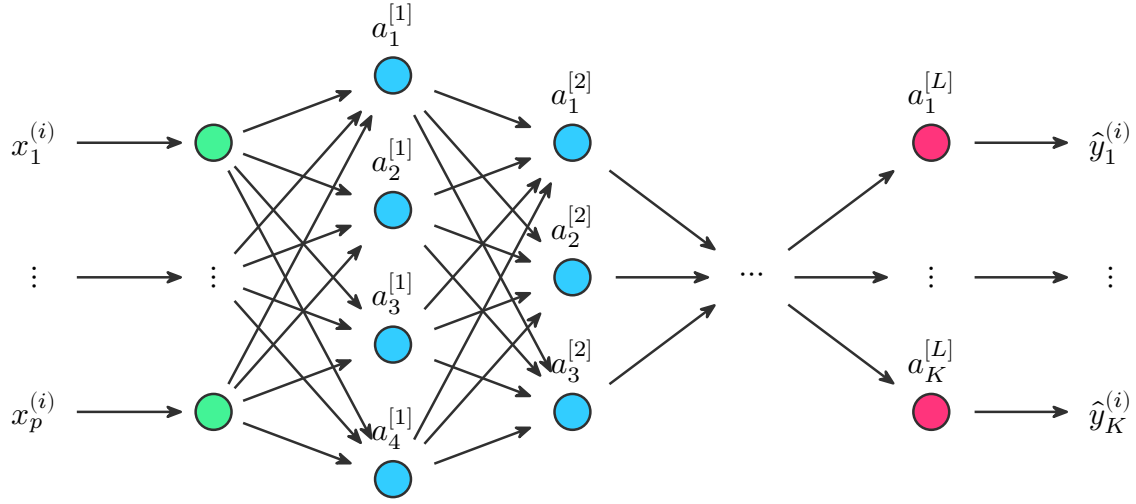
$$\hat{y}^{(i)} = a = g \left(\sum_{j=1}^p w_j x_j^{(i)} + b \right)$$

for some activation function g , weights w_j , inputs $x_j^{(i)}$, and bias term b . For notational convenience we will denote the weighted sum as:

$$z^{(i)} = \sum_{j=1}^p w_j x_j^{(i)} + b.$$

5.1.2 Multilayer Perceptron

A **multilayer perceptron** (MLP) is a development of the perceptron that contains at least one hidden layer. In this model, we introduce *hidden layers* that are placed between the input and output layers. The neurons in the hidden layer are *fully connected*, meaning that each neuron in the previous layer is connected to each neuron in the next layer.



As this model consists of multiple layers with multiple neurons in each layer, we will denote the output of the k -th neuron in the ℓ -th layer as $a_k^{[\ell]}$. The output of this neuron is given by:

$$a_k^{[\ell]} = g^{[\ell]} \left(\sum_{j=1}^{n^{[\ell-1]}} w_{kj}^{[\ell]} a_j^{[\ell-1]} + b_k^{[\ell]} \right).$$

Here, $n^{[\ell]}$ is the number of neurons in the ℓ -th layer, $w_{kj}^{[\ell]}$ is the weight connecting the j -th neuron in the $(\ell - 1)$ -th layer to the k -th neuron in the ℓ -th layer, and $b_k^{[\ell]}$ is the bias term of the k -th neuron in the ℓ -th layer. Note that a neuron has a single bias term, and the activation function can be different for each layer. Once again, we can denote the weighted sum for the k -th neuron in the ℓ -th layer as:

$$z_k^{[\ell]} = \sum_{j=1}^{n^{[\ell-1]}} w_{kj}^{[\ell]} a_j^{[\ell-1]} + b_k^{[\ell]}.$$

It is also useful to understand the output of the neurons at the first and last layers. The output of the k -th neuron in the first hidden layer for the i -th sample $\mathbf{x}^{(i)}$ is given by:

$$a_k^{[1]} = g^{[1]} \left(\sum_{j=1}^p w_{kj}^{[1]} x_j^{(i)} + b_k^{[1]} \right).$$

The output of the k -th neuron in the output layer (or the L -th layer) is given by:

$$\hat{y}_k^{(i)} = a_k^{[L]} = g^{[L]} \left(\sum_{j=1}^{n^{[L-1]}} w_{kj}^{[L]} a_j^{[L-1]} + b_k^{[L]} \right).$$

5.1.3 Vectorised Form

For both notational convenience and parallelisation, we can represent the above equations in a vectorised form. Let us define the following quantities:

- $\mathbf{a}^{[\ell]} \in \mathbb{R}^{n^{[\ell]}}$ as the vector of activations in the ℓ -th layer:

$$\mathbf{a}^{[\ell]} = \begin{bmatrix} a_1^{[\ell]} \\ \vdots \\ a_{n^{[\ell]}}^{[\ell]} \end{bmatrix}.$$

- $\mathbf{W}^{[\ell]} \in \mathbb{R}^{n^{[\ell]} \times n^{[\ell-1]}}$ as the matrix of weights connecting the ℓ -th layer to the $(\ell - 1)$ -th layer:

$$\mathbf{W}^{[\ell]} = \begin{bmatrix} w_{11}^{[\ell]} & \cdots & w_{1n^{[\ell-1]}}^{[\ell]} \\ \vdots & \ddots & \vdots \\ w_{n^{[\ell]}1}^{[\ell]} & \cdots & w_{n^{[\ell]}n^{[\ell-1]}}^{[\ell]} \end{bmatrix}.$$

- $\mathbf{b}^{[\ell]} \in \mathbb{R}^{n^{[\ell]}}$ as the vector of bias terms in the ℓ -th layer:

$$\mathbf{b}^{[\ell]} = \begin{bmatrix} b_1^{[\ell]} \\ \vdots \\ b_{n^{[\ell]}}^{[\ell]} \end{bmatrix}.$$

- $\mathbf{z}^{[\ell]} \in \mathbb{R}^{n^{[\ell]}}$ as the vector of weighted sums in the ℓ -th layer:

$$\mathbf{z}^{[\ell]} = \begin{bmatrix} z_1^{[\ell]} \\ \vdots \\ z_{n^{[\ell]}}^{[\ell]} \end{bmatrix} = \mathbf{W}^{[\ell]} \mathbf{a}^{[\ell-1]} + \mathbf{b}^{[\ell]}.$$

The output of the ℓ -th layer is therefore given by:

$$\mathbf{a}^{[\ell]} = g^{[\ell]}(\mathbf{z}^{[\ell]}),$$

so that the output of the network is the composition of the output of every layer:

$$\hat{\mathbf{y}}^{(i)} = \mathbf{a}^{[L]} = g^{[L]}(\mathbf{W}^{[L]} g^{[L-1]}(\mathbf{W}^{[L-1]} g^{[L-2]}(\dots g^{[1]}(\mathbf{W}^{[1]} \mathbf{x}^{(i)} + \mathbf{b}^{[1]}) \dots) + \mathbf{b}^{[L-1]}) + \mathbf{b}^{[L]}).$$

6 Training Neural Networks

6.1 Loss Functions

A loss function must be chosen at the output layer to determine how a network penalises errors during training. Below is a table of common loss functions used to train neural networks.

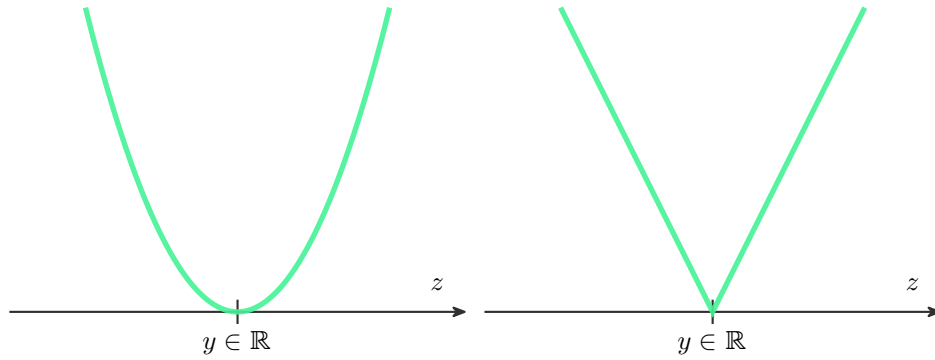
Task	Loss Function(s)
Regression	Mean Squared Error, Mean Absolute Error
Binary Classification	Binary Cross-Entropy (Logistic Loss), Hinge Loss
Multiclass Classification	Categorical Cross-Entropy, Sparse Categorical Cross-Entropy

The following graphs show the shape of common loss functions corresponding to a prediction z for a true value y .

Mean Squared Error & Mean Absolute Error Loss:

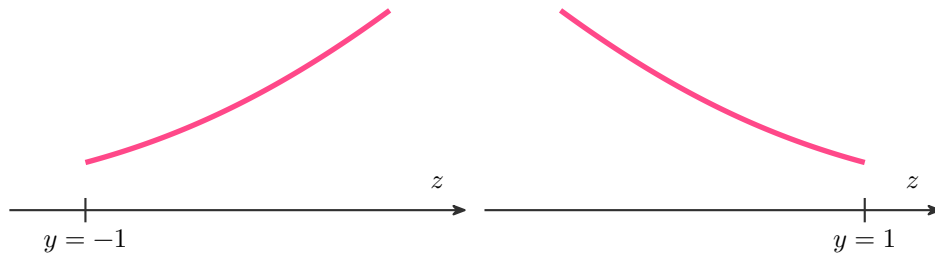
$$(y - z)^2 \quad \text{and} \quad |y - z|$$

for $y, z \in \mathbb{R}$.


Logistic Loss:

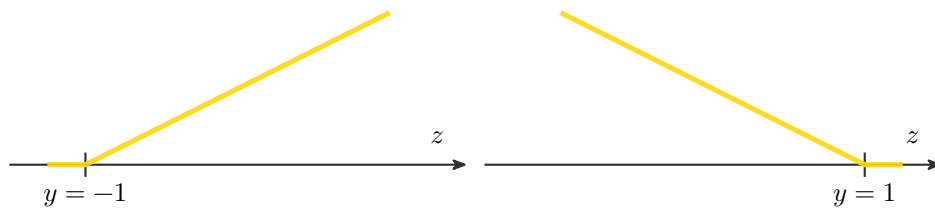
$$\ln(1 + \exp(-yz))$$

for $y \in \{-1, 1\}$ and $z \in \mathbb{R}$.


Hinge Loss:

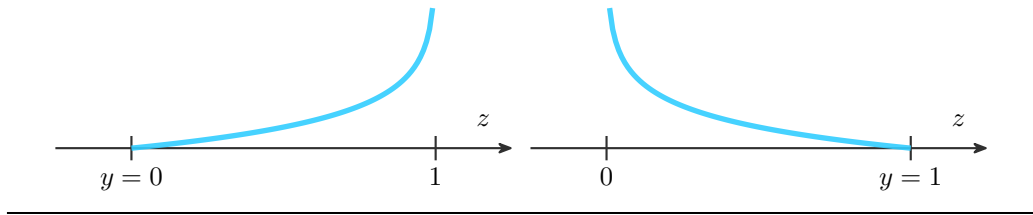
$$\max(0, 1 - yz)$$

for $y \in \{-1, 1\}$ and $z \in \mathbb{R}$.


Cross-Entropy Loss:

$$-[y \ln z + (1 - y) \ln(1 - z)]$$

for $y \in \{0, 1\}$ and $z \in [0, 1]$.



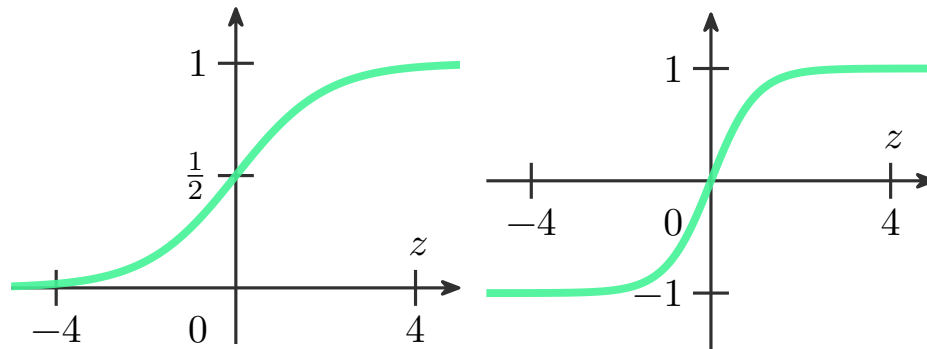
6.2 Activation Functions

Activation functions are used to introduce non-linearity into a neural network. If we simply passed weighted sums from one layer to the next, a neural network would be equivalent to a linear regression model, that is unable to learn complex patterns in data. Below is a table of common activation functions applied to the weighted sum z .

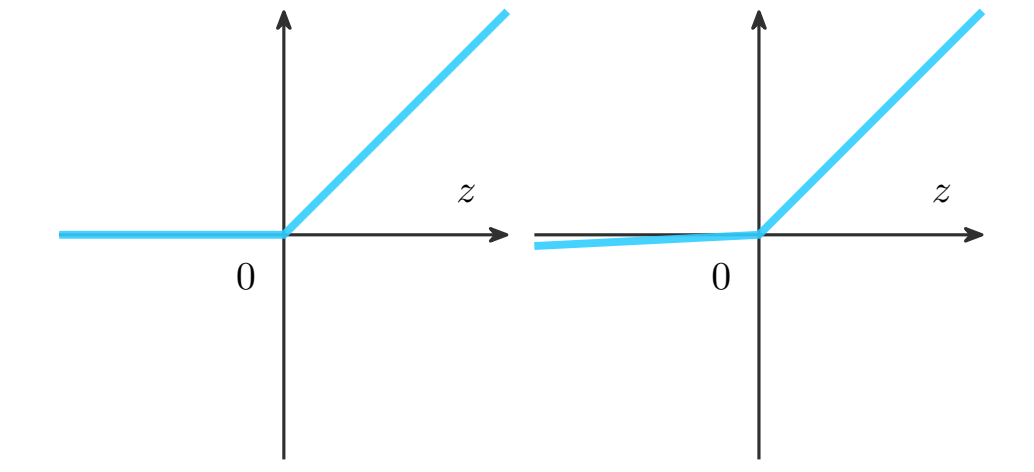
Activation Function	Formula
Sigmoid	$g(z) = \frac{1}{1 + \exp(-z)}$
Tanh	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
ReLU	$g(z) = \max(0, z)$
Leaky ReLU	$g(z) = \max(\epsilon z, z)$ for $\alpha \epsilon \ll 1$

These functions are plotted below:

Sigmoid & Tanh Functions:



ReLU & Leaky ReLU Functions:



The sigmoid and hyperbolic tangent functions are often used in the output layer of a neural network for binary classification tasks, as they produce values between 0 and 1, and -1 and 1, respectively. However, these functions are rarely used in deep neural networks, as they suffer from the **vanishing gradient problem**. During backpropagation, as the gradient of these functions is close to zero for large magnitudes of z , the resulting updates to the weights are very small, leading to a slow learning rate.

The ReLU function is a rectified linear unit, and is a very common function used to introduce non-linearity for its simplicity and ease of computation. The Leaky ReLU function is a variant of the ReLU function that allows a small gradient when the input is negative, which can help prevent the **dying ReLU problem** where neurons stop learning during training. The advantage of using a rectified function is that it also produces a sparse representation, that can help prevent overfitting.

6.3 Backpropagation

A neural network is trained by updating the weights of each neuron in each layer to minimise the loss at the output neuron. Let us derive an algorithm to efficiently update these weights in a feedforward neural network. Let us first define the **error** of a neuron in the L -th layer as:

$$\delta_k^{[L]} = \frac{\partial L}{\partial z_k^{[L]}},$$

then by using the multivariable chain rule, the error of a neuron in any subsequent layers $\ell < L$, is given by:

$$\delta_k^{[\ell]} = \frac{\partial L}{\partial z_k^{[\ell]}} = \sum_{l=1}^{n^{[\ell+1]}} \frac{\partial L}{\partial z_l^{[\ell+1]}} \frac{\partial z_l^{[\ell+1]}}{\partial z_k^{[\ell]}} = \sum_{l=1}^{n^{[\ell+1]}} \delta_l^{[\ell+1]} \frac{\partial z_l^{[\ell+1]}}{\partial z_k^{[\ell]}}.$$

Recall the definition of $z_l^{[\ell+1]}$:

$$z_l^{[\ell+1]} = \sum_{j=1}^{n^{[\ell]}} w_{lj}^{[\ell+1]} g^{[\ell]}(z_j^{[\ell]}) + b_l^{[\ell+1]} \implies \frac{\partial z_l^{[\ell+1]}}{\partial z_k^{[\ell]}} = w_{lk}^{[\ell+1]} g'^{[\ell]}(z_k^{[\ell]}).$$

Therefore, the error of a neuron in the ℓ -th layer is given by the iterative formula:

$$\delta_k^{[\ell]} = \sum_{l=1}^{n^{[\ell+1]}} \delta_l^{[\ell+1]} w_{lk}^{[\ell+1]} g'^{[\ell]}(z_k^{[\ell]}) = g'^{[\ell]}(z_k^{[\ell]}) \sum_{l=1}^{n^{[\ell+1]}} \delta_l^{[\ell+1]} w_{lk}^{[\ell+1]}.$$

Using the chain rule, we can then compute the gradient of the loss function with respect to the weights in the ℓ -th layer:

$$\frac{\partial L}{\partial w_{kj}^{[\ell]}} = \frac{\partial L}{\partial a_k^{[\ell]}} \frac{\partial a_k^{[\ell]}}{\partial z_k^{[\ell]}} \frac{\partial z_k^{[\ell]}}{\partial w_{kj}^{[\ell]}} = \delta_k^{[\ell]} a_j^{[\ell-1]} = g'^{[\ell]}(z_k^{[\ell]}) a_j^{[\ell-1]} \sum_{l=1}^{n^{[\ell+1]}} \delta_l^{[\ell+1]} w_{lk}^{[\ell+1]}$$

where the last term is simply the activation of the previous layer:

$$\frac{\partial z_k^{[\ell]}}{\partial w_{kj}^{[\ell]}} = \frac{\partial}{\partial w_{kj}^{[\ell]}} \left(\sum_{l=1}^{n^{[\ell-1]}} w_{kl}^{[\ell]} a_l^{[\ell-1]} + b_k^{[\ell]} \right) = a_j^{[\ell-1]}.$$

This is known as **backpropagation**, as the error from the output layer is propagated backwards through the network to compute the gradients of neurons in previous layers. A similar formula can be derived for the bias terms:

$$\frac{\partial L}{\partial b_k^{[\ell]}} = \frac{\partial L}{\partial a_k^{[\ell]}} \frac{\partial a_k^{[\ell]}}{\partial z_k^{[\ell]}} \frac{\partial z_k^{[\ell]}}{\partial b_k^{[\ell]}} = \delta_k^{[\ell]} = g'^{[\ell]}(z_k^{[\ell]}) \sum_{l=1}^{n^{[\ell+1]}} \delta_l^{[\ell+1]} w_{lk}^{[\ell+1]}.$$

The gradient descent update rule for the weights and biases is then:

$$\begin{aligned} w_{kj}^{[\ell]} &\leftarrow w_{kj}^{[\ell]} - \alpha \frac{\partial L}{\partial w_{kj}^{[\ell]}} = w_{kj}^{[\ell]} - \alpha \delta_k^{[\ell]} a_j^{[\ell-1]} \\ b_k^{[\ell]} &\leftarrow b_k^{[\ell]} - \alpha \frac{\partial L}{\partial b_k^{[\ell]}} = b_k^{[\ell]} - \alpha \delta_k^{[\ell]} \end{aligned}$$

for a learning rate α , where

$$\delta_k^{[\ell]} = \begin{cases} g'^{[\ell]}(z_k^{[\ell]}) \sum_{l=1}^{n^{[\ell+1]}} \delta_l^{[\ell+1]} w_{lk}^{[\ell+1]}, & \ell < L \\ \frac{\partial L}{\partial z_k^{[\ell]}}, & \ell = L \end{cases}$$

The **backpropagation algorithm** can be described with the following steps:

1. During the **forward phase**, the program stores:
 - The predictions $\hat{y}^{(i)}$.
 - The weighted sums $z_k^{[\ell]}$ for each layer.
 - The output of each neuron $a_k^{[\ell]}$ for each layer.

for each observation $\mathbf{x}^{(i)}$.

2. During the **backward phase**:

- (a) Compute the error of the output layer $\delta_k^{[L]}$.
- (b) Backpropagate the final layer error through the network to compute the error of each neuron in each layer $\delta_k^{[\ell]}$.
- (c) Compute the gradient of the loss function with respect to the weights and biases, $w_{kj}^{[\ell]}$ and $b_k^{[\ell]}$, using the equations derived above.

3. Combine the losses over a batch of observations to calculate the cost.

4. Update the weights and biases using an optimisation algorithm.

6.4 Vectorised Backpropagation

We can vectorise the equations above by denoting the vector of errors in the ℓ -th layer with $\boldsymbol{\delta}^{[\ell]} \in \mathbb{R}^{1 \times n^{[\ell]}}$, defined as:

$$\boldsymbol{\delta}^{[\ell]} = \left(\frac{\partial \ell}{\partial \mathbf{z}^{[\ell]}} \right)^\top$$

The gradient descent update rule then becomes:

$$\begin{aligned} \mathbf{W}^{[\ell]} &\leftarrow \mathbf{W}^{[\ell]} - \alpha \boldsymbol{\delta}^{[\ell]} (\mathbf{a}^{[\ell-1]})^\top \\ \mathbf{b}^{[\ell]} &\leftarrow \mathbf{b}^{[\ell]} - \alpha \boldsymbol{\delta}^{[\ell]} \end{aligned}$$

with the error of a neuron in the ℓ -th layer given by:

$$\boldsymbol{\delta}^{[\ell]} = \begin{cases} g'^{[\ell]} (\mathbf{z}^{[\ell]})^\top \odot (\boldsymbol{\delta}^{[\ell+1]} \mathbf{W}^{[\ell+1]}), & \ell < L \\ \left(\frac{\partial L}{\partial \mathbf{z}^{[\ell]}} \right)^\top, & \ell = L \end{cases}$$

for $\ell < L$, where $\odot$ denotes the Hadamard product (element-wise product).

6.5 Batch Backpropagation

We can further optimise this algorithm by parallelising the computation of gradients for a batch of observations, through the use of matrix operations. So far we have implicitly assumed a single observation $\mathbf{x}^{(i)}$ in all our formulations:

$$\mathbf{b}^{[\ell]} = \mathbf{b}^{[\ell](i)}, \quad \mathbf{w}^{[\ell]} = \mathbf{w}^{[\ell](i)}, \quad \mathbf{z}^{[\ell]} = \mathbf{z}^{[\ell](i)}, \quad \mathbf{a}^{[\ell]} = \mathbf{a}^{[\ell](i)}, \quad \boldsymbol{\delta}^{[\ell]} = \boldsymbol{\delta}^{[\ell](i)}.$$

Let us define the following matrices to hold the information of b observations:

- $\mathbf{X} \in \mathbb{R}^{p \times b}$ as the matrix of input observations:

$$\mathbf{X} = \begin{bmatrix} | & & | \\ \mathbf{x}^{(1)} & \dots & \mathbf{x}^{(b)} \\ | & & | \end{bmatrix}.$$

- $\mathbf{Y} \in \mathbb{R}^{K \times b}$ as the matrix of target observations:

$$\mathbf{Y} = \begin{bmatrix} | & & | \\ \mathbf{y}^{(1)} & \dots & \mathbf{y}^{(b)} \\ | & & | \end{bmatrix}.$$

- $\mathbf{A}^{[\ell]} \in \mathbb{R}^{n^{[\ell]} \times b}$ as the matrix of activations in the ℓ -th layer:

$$\mathbf{A}^{[\ell]} = \begin{bmatrix} | & & | \\ \mathbf{a}^{[\ell](1)} & \dots & \mathbf{a}^{[\ell](b)} \\ | & & | \end{bmatrix}.$$

- $\mathbf{Z}^{[\ell]} \in \mathbb{R}^{n^{[\ell]} \times b}$ as the matrix of weighted sums in the ℓ -th layer:

$$\mathbf{Z}^{[\ell]} = \begin{bmatrix} | & & | \\ \mathbf{z}^{[\ell](1)} & \dots & \mathbf{z}^{[\ell](b)} \\ | & & | \end{bmatrix} = \mathbf{W}^{[\ell]} \mathbf{A}^{[\ell-1]} + \mathbf{B}^{[\ell]}.$$

- $\mathbf{B}^{[\ell]} \in \mathbb{R}^{n^{[\ell]} \times b}$ as the matrix of bias terms in the ℓ -th layer:

$$\mathbf{B}^{[\ell]} = \begin{bmatrix} | & & | \\ \mathbf{b}^{[\ell]} & \dots & \mathbf{b}^{[\ell]} \\ | & & | \end{bmatrix}.$$

Note that we must broadcast the bias terms b times across the columns of this matrix for the above matrix addition to be valid.

- $\Delta^{[\ell]} \in \mathbb{R}^{n^{[\ell]} \times b}$ as the matrix of errors in the ℓ -th layer:

$$\Delta^{[\ell]} = \begin{bmatrix} | & & | \\ \delta^{[\ell](1)} & \dots & \delta^{[\ell](b)} \\ | & & | \end{bmatrix}.$$

Thus the output of the network is given by:

$$\hat{\mathbf{Y}} = \mathbf{A}^{[L]} = g^{[L]}(\mathbf{Z}^{[L]}).$$

For a batch of b observations, the update rule for the weights and biases in the ℓ -th layer is now:

$$\begin{aligned} \mathbf{W}^{[\ell]} &\leftarrow \mathbf{W}^{[\ell]} - \alpha \frac{1}{n} \Delta^{[\ell]} (\mathbf{A}^{[\ell-1]})^\top \\ \mathbf{B}^{[\ell]} &\leftarrow \mathbf{B}^{[\ell]} - \alpha \frac{1}{n} \Delta^{[\ell]} \end{aligned}$$

where the error of neurons in the ℓ -th layer is given by:

$$\Delta^{[\ell]} = \begin{cases} g'^{[\ell]}(\mathbf{Z}^{[\ell]})^\top \odot (\Delta^{[\ell+1]} \mathbf{W}^{[\ell+1]}), & \ell < L \\ \left(\frac{\partial L}{\partial \mathbf{Z}^{[\ell]}} \right)^\top, & \ell = L \end{cases}$$

6.6 Considerations for Classification Problems

6.6.1 Target Variable Encoding

The choice of loss function depends on the encoding of the target variable.

- **One-hot encoding** is used when each class is represented as a unique binary vector where all elements are zero except for the element corresponding to the class.
- **Integer encoding** is used when each class is represented as a unique integer.

If the target variable is one-hot encoded, then the **categorical cross-entropy** loss function is used. If classes in the target variable are encoded as integers, then the **sparse categorical cross-entropy** loss function is used.

6.6.2 Output Layer

Software packages that implement neural networks allow the user to specify the output of a network in two ways:

- **Probabilities**: The output of the network is a probability distribution over the classes. This is often achieved by applying a **softmax** activation function to the output layer, so that values range between 0 and 1.
- **Logits**⁴: The output of the network is the raw output of the network before the softmax activation function is applied, so that values range between $-\infty$ and ∞ . This is often useful when the network is used for inference.

6.7 Ensemble Methods

During training, neural networks introduce randomness in the form of initialisation of weights, dropout, data shuffling, etc. This randomness often produces different models that eventually converge after many epochs. We can leverage each models results by creating an ensemble of models. Here we can choose many ways to combine the results of each model, such as through averaging. Note that while model performance may increase, the computational cost of training and inference will also increase significantly.

7 Optimisation Algorithms and Regularisation

The following sections discuss various techniques used to solve nonlinear optimisation problems. The optimisation problems relevant to machine learning are minimisation problems that aim to minimise the loss function of a given learning method. The following sections discuss several iterative algorithms that successively update the weights of a model using the gradient of the loss function.

⁴The word “logit” comes from **logistic unit**, and is defined as the inverse of the sigmoid function. The same term is used in the context of the softmax function.

7.1 Gradient Descent

Gradient descent is the most basic optimisation algorithm, used to minimise a loss function. The algorithm works by taking steps in the opposite direction of the gradient of the loss function, which is the direction of greatest decrease. The resulting update rule is given by:

$$w \leftarrow w - \alpha \frac{\partial L(w)}{\partial w}$$

for a learning rate α . This learning rate is a hyperparameter that controls the size of each step. If the learning rate is too small, the algorithm may take a long time to converge, and if it is too large, the algorithm may oscillate around a minimum.

7.2 Stochastic Gradient Descent

There are several variants of gradient descent that are used to improve convergence and speed up training.

- **Batch Gradient Descent:** Weights are updated after computing the gradient of the loss function on the entire training set:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}, \quad J(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n L^{(i)}(\mathbf{w}).$$

This update rule is performed 1 time (1 update per epoch).

- **Mini-Batch Gradient Descent:** Weights are updated after computing the gradient of the loss function on a small batch of the training set:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}}, \quad J(\mathbf{w}) = \frac{1}{b} \sum_{i=1}^b L^{(i)}(\mathbf{w})$$

for a batch size b . This update rule is performed $\lceil n/b \rceil$ times ($\lceil n/b \rceil$ updates per epochs).

- **Stochastic Gradient Descent:** Weights are updated after computing the gradient of the loss function on a single training example (batch size of 1):

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial L^{(i)}(\mathbf{w})}{\partial \mathbf{w}}$$

for $1 \leq i \leq n$. This update rule is performed n times (n updates per epochs).

7.2.1 Epochs

Each update step will be performed repeatedly until the entire training set has been considered. An epoch represents the time taken to apply gradient descent on the entire training set. The number of updates in an epoch is the number of batches required to cover the entire training set. That is,

$$\text{updates per epoch} = \frac{n}{b}.$$

When setting an upper bound on the number of epochs, we should consider the following:

- Does the validation accuracy continue to increase after the current epoch?
- Will the training set perform better than the validation set after the current epoch, leading to overfitting?

7.2.2 Batch Size

Choosing a small batch size will lead to a greater number of updates per epoch, but result in a suboptimal path toward a minimum, as each batch is less representative of the overall dataset, and therefore the average gradient will not point toward the direction of greatest descent. A small batch size may also lead to underfitting if classes are unbalanced.

Choosing a large batch size will lead to a more direct path toward a local minimum, but may require more epochs to reach a satisfactory result. A large batch size may also result in higher memory usage.

7.3 Optimisers

Several variants of gradient descent have been proposed to improve the convergence and performance of the gradient descent algorithm. The following sections discuss how some optimisers use adaptive learning rates and parameter updates based on previous gradients to navigate the parameter space more effectively.

7.3.1 Momentum

The momentum optimiser computes an intermediate velocity vector $\mathbf{v}$ that depends on the previous velocity of the update rule and the gradient of the weights. This optimiser aims to smooth the updates made by the stochastic gradient descent algorithm by keeping the momentum of previous updates. The momentum update rule is shown below:

$$\begin{aligned}\mathbf{v} &\leftarrow m\mathbf{v} - \alpha \frac{\partial L^{(i)}(\mathbf{w})}{\partial \mathbf{w}} \\ \mathbf{w} &\leftarrow \mathbf{w} + \mathbf{v}\end{aligned}$$

where $0 \leq m \leq 1$ is the momentum and α is the learning rate.

7.3.2 Nesterov Accelerated Gradient (NAG)

A method closely related to the momentum method is Nesterov Accelerated Gradient (NAG). The aim of NAG is to correct oscillations in the momentum method by computing an intermediate update $\mathbf{w}'$ using the momentum from the previous iteration, and correcting this update with the gradient of the weights at $\mathbf{w}'$. The NAG update rule is shown below:

$$\begin{aligned}\mathbf{w}' &\leftarrow \mathbf{w} - m\mathbf{v} \\ \mathbf{v} &\leftarrow \alpha \frac{\partial L^{(i)}(\mathbf{w}')}{\partial \mathbf{w}} \\ \mathbf{w} &\leftarrow \mathbf{w} - \mathbf{v}\end{aligned}$$

where $0 \leq m \leq 1$ is the momentum and α is the learning rate. This allows the algorithm to correct a high loss momentum update after it has been made.

7.3.3 Resilient Propagation (RProp)

In some cases, the gradient of the loss function can either be extremely small, leading to slow convergence, or extremely large, leading to oscillatory behaviour around minima, and possibly divergence. To combat this, let us instead consider only the sign of the gradient and update the learning rate instead.

$$\alpha \leftarrow \begin{cases} \min(\alpha^+ \alpha, \alpha_{\max}), & \frac{\partial L^{(i)}(\mathbf{w})}{\partial \mathbf{w}} \mathbf{g} > 0 \\ \max(\alpha^- \alpha, \alpha_{\min}), & \frac{\partial L^{(i)}(\mathbf{w})}{\partial \mathbf{w}} \mathbf{g} < 0 \\ \alpha, & \text{otherwise} \end{cases}$$

$$\mathbf{g} \leftarrow \frac{\partial L^{(i)}(\mathbf{w})}{\partial \mathbf{w}}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \odot \text{sgn}(\mathbf{g})$$

In the first step, we update α so that it is between $\alpha^- < 1 < \alpha^+$, imposing the lower and upper bounds $\alpha_{\min}$ and $\alpha_{\max}$ to ensure that the learning rate does not become too small or too large. This choice depends on the product of the gradient of the loss function at the current step $\frac{\partial L^{(i)}(\mathbf{w})}{\partial \mathbf{w}}$ and the gradient of the loss function at the previous step $\mathbf{g}$.

$\mathbf{g}$ is then updated with the current gradient, and the weights are updated using the adaptive learning and sign of $\mathbf{g}$. Note that the min, max, and sgn functions are applied element-wise.

7.3.4 Adaptive Gradient (AdaGrad)

7.3.5 RMSprop

$$\mathbf{s} \leftarrow \beta \mathbf{s} + (1 - \beta) (\nabla J(\mathbf{w}) \odot \nabla J(\mathbf{w}))$$

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \left(\frac{\nabla J(\mathbf{w})}{\sqrt{\mathbf{s} + \epsilon}} \right)$$

7.3.6 Adam

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla J(\mathbf{w})$$

$$\mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) (\nabla J(\mathbf{w}) \odot \nabla J(\mathbf{w}))$$

$$\hat{\mathbf{m}} \leftarrow \frac{\mathbf{m}}{1 - \beta_1^t}$$

$$\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \beta_2^t}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \left(\frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{s}} + \epsilon}} \right)$$

7.4 Regularisation

Deep neural networks often result in a large number of parameters that exceed the number of samples used to train the model. This makes it very easy for the model to overfit the training data. In such cases, we can either:

- Modify the network to reduce overfitting
- Increase the number of training samples

7.4.1 Dropout Layers

Dropout layers allow us to randomly disconnect some neurons in the network during training to perform each backpropagation step on a different network. This helps to prevent the network from overfitting the training data at the cost of increased training time.

7.4.2 Batch Normalisation

To allow each layer to train on the same scale upon each update, we can apply a batch normalisation layer to the network which normalises the output of the previous layer to have a mean of 0 and a standard deviation of 1 (although the mean and standard deviation can be learned also). This layer should be placed before any activation functions in the network to make it easier to consider the impact of the proceeding layer and improve training speed. Note that batch normalisation layers do not need to be used very frequently.

7.4.3 Weight Regularisation

As with linear regression, we want to ensure that weights do not become too large. We can therefore use weight regularisation to penalise large weights in a network as a sum of the weights. This can be done using **L1** or **L2** regularisation (or both), where we may choose to apply this penalty on weights, biases, or activation functions.

7.4.4 Fine Tuning

Fine tuning is the process of taking a pre-trained model and training it on a new dataset. This is often done by freezing some percentage of weights in the earlier layers and training the remaining layers in addition to new layers that were added specifically for the new task. This is useful when the dataset is very small compared to the number of parameters in the network, and allows us to reduce the training time of large networks by taking advantage of similar features learned in the pre-trained model.

7.4.5 Augmentation

To avoid overfitting, we can increase the size of the training set by applying transformations to the data. This is known as data augmentation. These transformations can include:

- Rotating the image
- Flipping the image
- Cropping the image
- Scaling the image
- Translating the image

- Changing the brightness of the image
- Adding noise or colour shifts to the image

Note that augmentation should not change the meaning of an image, nor should it be too extreme that such an image would not be found in the test dataset.

8 Convolutional Neural Networks

A convolutional neural network (CNN) is a type of neural network that is used to process spatial data, such as images. CNNs are characterised by their use of convolutional layers, which apply a series of filters to images to extract features.

8.1 Convolutional Layers

A convolutional layer consists of convolution operations defined:

$$\mathbf{G}_{xy} = \boldsymbol{\omega} * \mathbf{F} = \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \sum_{k=1}^c \omega_{ij} \mathbf{F}_{x-\lfloor h/2 \rfloor+i, y-\lfloor w/2 \rfloor+j, k}$$

where $\mathbf{F}$ is an image of dimensions $w \times h \times c$, $\boldsymbol{\omega}$ is a kernel of dimensions $W \times H$, and $\mathbf{G}$ is the output which has dimensions $w \times h \times n$, where n is the number of filters in the convolutional layer.

The choice of kernel determines the kind of features that are extracted from an image. When used in a neural network, the weights of this kernel are learned during training, allowing the network to potentially learn several features at once. For an image with colour channels, the filter is applied to each channel separately, and the results are summed to produce a single output. This results in the following number of parameters:

$$\text{number of parameters} = (w \times h \times c + 1) \times n.$$

Note that the $+1$ term accounts for the bias term in the convolutional layer. As this kernel is variable in size, the convolution along the border of an image may exceed the dimensions of the image. In such cases, we can choose to:

- Pad the image with 0, 1, or -1 to maintain the dimensions of the image.
- Apply the convolution operation only to the pixels that are fully covered by the kernel, resulting in a smaller output.

We can also decide to apply a stride to the convolution operation, which determines the number of pixels the kernel moves across the image.

8.2 Pooling Layers

Pooling layers are used to reduce the dimensions of the outputs produced by convolutional layers, and therefore reduce the number of parameters in the network. Pooling layers are placed after convolutional layers and are used to aggregate features in one of the following ways:

- **Max Pooling:** The maximum value in a region is selected as the output. This is the preferred type of pooling.
- **Min Pooling:** The minimum value in a region is selected as the output.
- **Average Pooling:** The average value in a region is selected as the output.

The size of the pooling region determines the size of the output.

8.3 Deep Convolutional Neural Networks

Convolutional neural networks are most effective when multiple layers are stacked on top of each other. This allows the network to learn increasingly complex features from the input data. Often this is done with small kernels of size 3×3 or 5×5 , and each convolutional layer is followed by a pooling layer to reduce the dimensions of the output.

8.3.1 VGG Networks

The Visual Geometry Group (VGG) network, is one such network that primarily consists of stacked convolutional layers followed by max pooling layers, all repeated, with a few fully connected layers near the end. While increasing the number of convolutions allows the network to learn complex features, it also becomes increasingly difficult to train the network. This is because data now has to pass through many layers before reaching the output, causing small changes in early layers to have significant impacts on later layers. Additionally, backpropagation may result in the gradient vanishing or exploding as it passes through many layers, causing the network to stop learning beyond a certain number of layers. Therefore, we need a more direct connection from the input to the output of a network.

8.3.2 Residual Networks

To combat this, we can use residual networks, which introduce skip connections that allow the network to learn the difference or sum of the original input and the output of convolutional layers, allowing the network to give meaningful feedback to earlier layers. When the number of layers is too large, we can also introduce bottleneck layers that reduce the number of parameters in the network. This is done with a 1×1 convolutional layer that reduces the number of channels in the input, followed by a 3×3 convolutional layer, and finally a 1×1 convolutional layer that increases the number of channels back to the original number.

Part III

Unsupervised Learning

9 Clustering

Clustering is an unsupervised learning method used to discover patterns in data. The goal of a clustering algorithm is to group similar data into clusters allowing us to identify patterns or abnormalities in the data.

9.1 K-Means Clustering

K-means clustering aims to find a set of k clusters that minimise the distances between data points and their cluster centres. The algorithm works by iteratively updating the cluster centres and reassigning data points to the nearest cluster. The algorithm is summarised below:

1. Choose k initial cluster centres.
2. Assign each data point to the nearest cluster.
3. Update the cluster centres by taking the mean of the data points in each cluster.
4. Repeat steps 2 and 3 until the cluster centres converge.

As these clusters are sensitive to the initialisation of the cluster centres, it is often useful to run the algorithm multiple times with different initialisations and choose the best result. These differences become more pronounced as the number of data points and clusters increases.

The distance metric used in this algorithm depends entirely on the data. When the dimensions of the data are not on the same scale, it is often useful to normalise or standardise the data before running the algorithm, so that no single dimension dominates the distance calculation. Note that because of this, the algorithm is restricted to roughly spherical clusters.

9.2 Gaussian Mixture Models

Rather than assigning a data point to a single cluster, Gaussian Mixture Models (GMMs) assign a probability that a data point belongs to a cluster.

9.3 Model Selection

When choosing the number of clusters in the above algorithms, we can use one of two metrics⁵:

- **Reconstruction Error:** Average distance to assigned cluster centre:

$$J(c, \mu) = \sum_{i=1}^n \|\mathbf{x}^{(i)} - \mu_{c^{(i)}}\|^2.$$

This metric is minimised when the number of clusters is equal to the number of data points.

- The optimal number of clusters is often the point at which the rate of decrease in the reconstruction error slows down. This is known as the **elbow point** of the curve.
- **Bayesian Information Criterion (BIC):** A metric that captures how informative a model is, while considering model complexity. An approximation of the BIC for k -means is given by:

$$\text{BIC} = n \log \sum_{i=1}^n \|\mathbf{x}^{(i)} - \mu_{c^{(i)}}\|^2 + k \log n.$$

- The optimal number of clusters is the number of clusters that minimises the BIC.

The choice of k should be chosen with care to avoid under-clustering, where true clusters are merged into a single cluster, or over-clustering, where a true cluster is divided into multiple sub-clusters.

⁵Here $c^{(i)}$ is the cluster assignment of the i -th data point, and $\mu_{c^{(i)}}$ is the cluster centre of the assigned cluster.

10 Dimensionality Reduction

Dimensionality reduction is a technique used to reduce the number of features in a dataset. This is useful when the number of features is very large and the data is sparse, as it can help to reduce the computational cost of training a model by removing less important features.

10.1 Principal Component Analysis

Principal Component Analysis (PCA) is an unsupervised technique used to reduce the dimensionality of a dataset by projecting the data onto a lower dimensional space that preserves as much of the variance in the data as possible. Note that we must first standardise the data before applying PCA, and ensure that the number of samples is greater than the number of features.

10.2 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is a supervised method used to find a projection that best discriminates between classes in a dataset. It considers the variance of the data within each class, as well as the variance of the data between classes, and aims to maximise the ratio of these two variances.

LDA returns $k - 1$ components.

Note that LDA is sensitive to the number of samples per class, so it is important to ensure that each class has more samples than the number of features in the dataset. Having too many features or classes may also lead to poor performance. In particular, the scatter matrices used in this algorithm may become close to singular and lead to overfitting. In such cases, it may be useful to first apply PCA before applying LDA.

Unlike PCA, LDA also cannot be inverted to reconstruct the original data, as it removes information from the data.

10.3 t-Distributed Stochastic Neighbour Embedding

t-Distributed Stochastic Neighbour Embedding (t-SNE) is a technique used to visualise high-dimensional data in a 2-dimensional space. It uses local relationships to create a low-dimensional mapping that can capture the non-linear structure of the data.

11 Autoencoders

12 Sequences and Attention Mechanisms

A Numerical Summaries of Data

A.1 Mean

The mean of a set of n observations $(x^{(i)})$ is given by:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x^{(i)}$$

A.2 Variance

The variance of a set of n observations $(x^{(i)})$ is given by:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x^{(i)} - \bar{x})^2$$

For a sample, the variance is given by:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x^{(i)} - \bar{x})^2$$

A.3 Covariance

The covariance between two sets of n observations $(x^{(i)})$ and $(y^{(i)})$ is given by:

$$\sigma_{xy} = \frac{1}{n} \sum_{i=1}^n (x^{(i)} - \bar{x}) (y^{(i)} - \bar{y})$$

For a sample, the covariance is given by:

$$s_{xy} = \frac{1}{n-1} \sum_{i=1}^n (x^{(i)} - \bar{x}) (y^{(i)} - \bar{y})$$